



哈爾濱工業大學

HARBIN INSTITUTE OF TECHNOLOGY



分布式机器学习(IV)

数据并行和无重复计算

王宏志

哈尔滨工业大学

计算学部海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

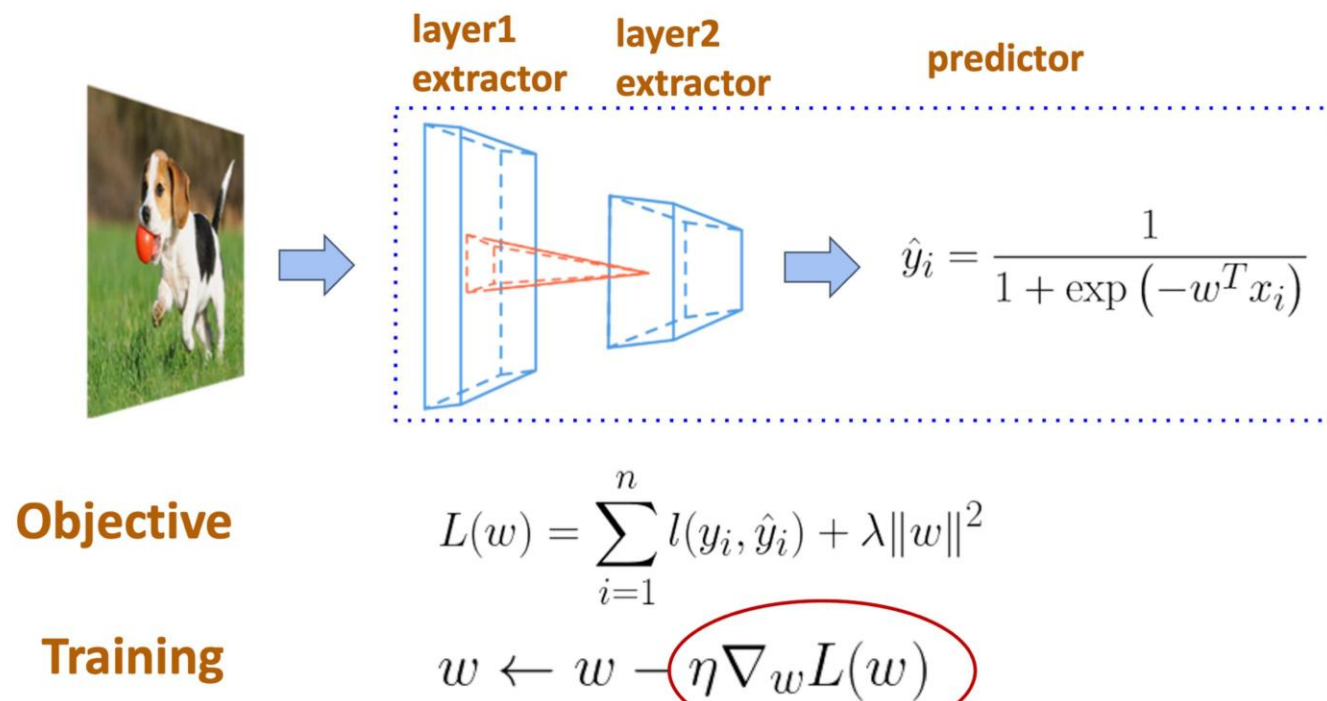
- 1 分布式深度学习训练
- 2 数据并行与梯度聚合
- 3 大模型训练的内存挑战
- 4 ZeRO与全分片数据并行

目录

Contents

- 1 分布式深度学习训练
- 2 数据并行与梯度聚合
- 3 大模型训练的内存挑战
- 4 ZeRO与全分片数据并行

深度神经网络训练概述

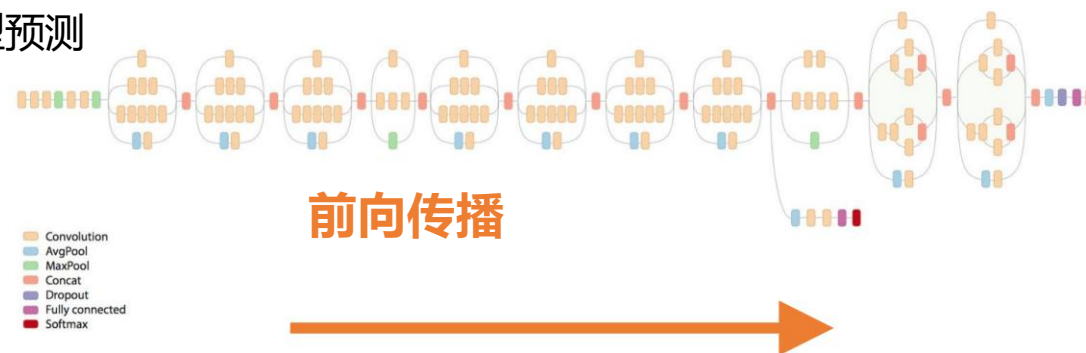


深度神经网络训练过程

通过3个阶段的多次迭代来训练ML模型

- 1.前向传播:将模型应用于一批输入样本,并通过算子运行计算以产生预测
- 2.向后传播:反向运行模型,使每个可训练的权重产生误差
- 3.权重更新:使用损失值更新模型权重

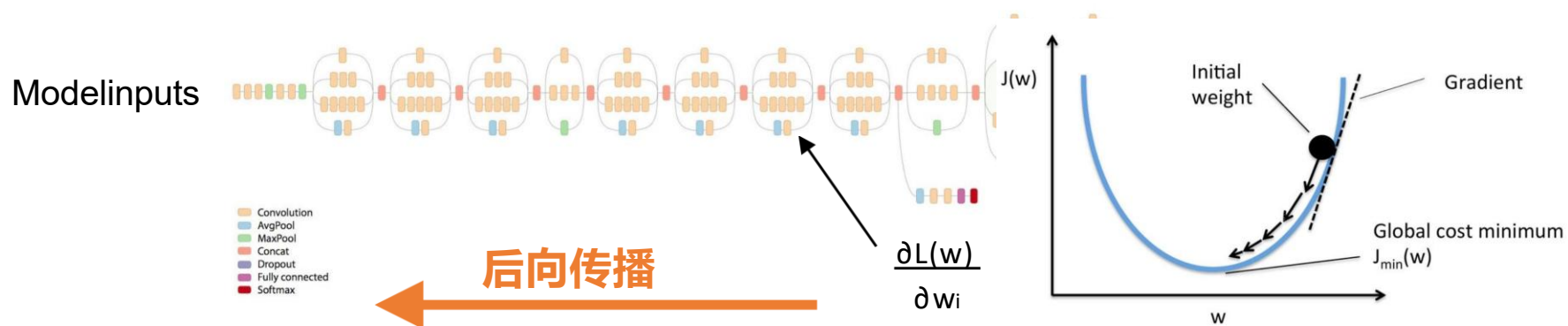
模型输入模型预测



深度神经网络训练过程

通过3个阶段的多次迭代来训练ML模型

- 1.前向传播:将模型应用于一批输入样本,并通过算子运行计算以产生预测
- 2.向后传播:倒运行模型,为每个可训练的权重生成一个梯度
- 3.权重更新:使用损失值更新模型权重



深度神经网络训练过程

通过3个阶段的多次迭代来训练ML模型

- 1.前向传播:将模型应用于一批输入样本,并通过算子运行计算以产生预测
- 2.向后传播:倒运行模型,为每个可训练的权重生成一个梯度
- 3.权重更新:使用损失值更新模型权重

$$w_i := w_i - \gamma \frac{\partial L(w)}{\partial w_i} = w_i - \frac{\gamma}{n} \sum_{j=1}^n \boxed{\frac{\partial l_i(w)}{\partial w_i}} \quad \text{单个样本的梯度}$$

如何并行DNN训练？

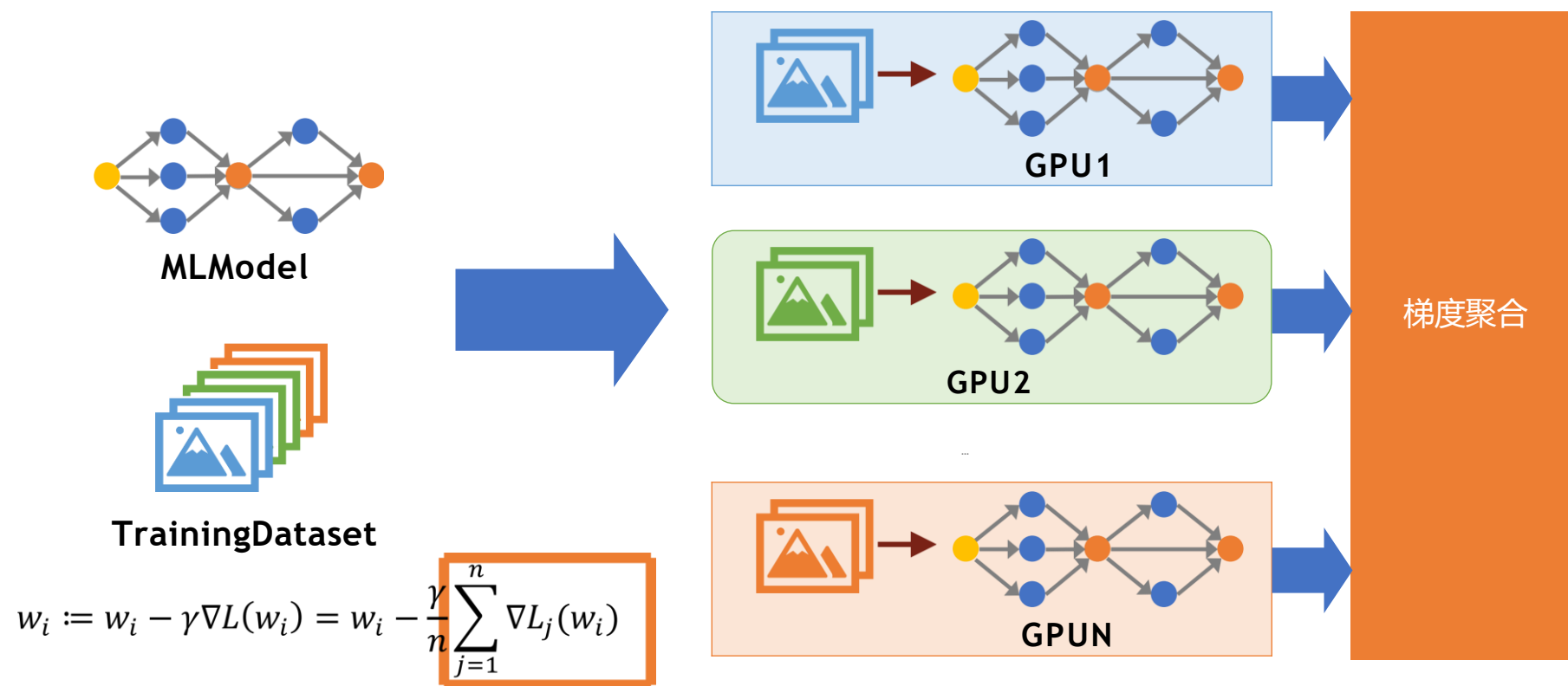
$$w_i := w_i - \gamma \nabla L(w_i) = w_i - \frac{\gamma}{n} \sum_{j=1}^n \nabla L_j(w_i)$$

目录

Contents

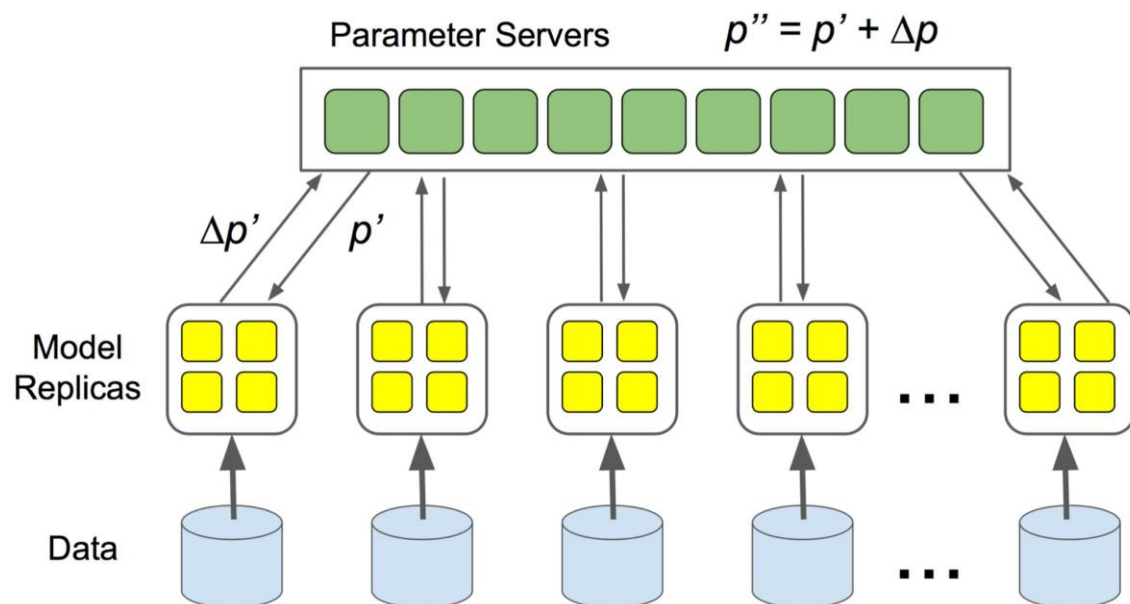
- 1 分布式深度学习训练
- 2 数据并行与梯度聚合**
- 3 大模型训练的内存挑战
- 4 ZeRO与全分片数据并行

数据并行



1. 将训练数据划分为batch
2. 计算GPU上每批的梯度
3. 跨GPU聚合梯度

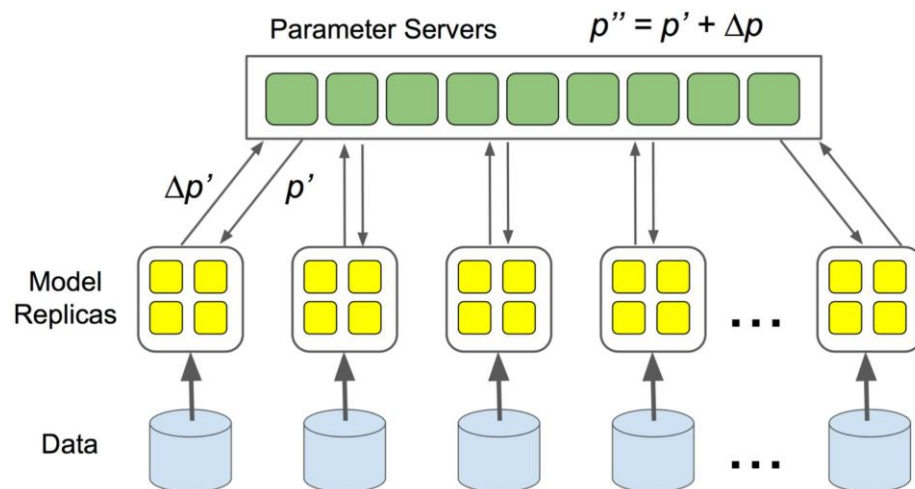
数据并行：参数服务器



工作节点将梯度推送到参数服务器,并拉回更新的参数

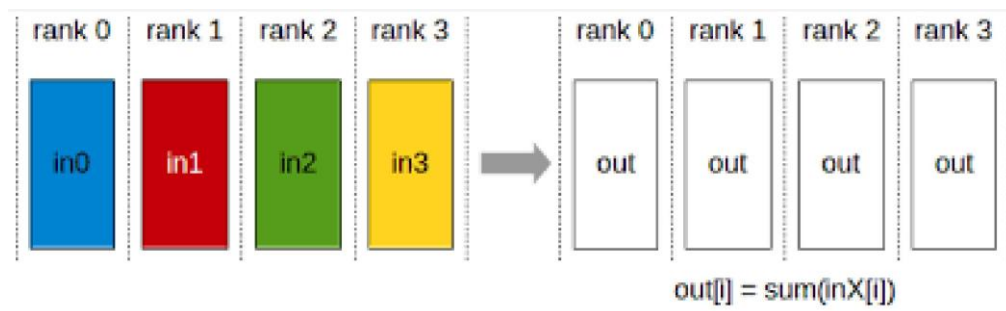
参数服务效率低下

- 集中通信:所有工作节点都与参数服务器通信以更新权重;不能扩展到大量工作节点
- 如何在DNN训练中分散通信?



参数服务器效率低下

- 集中通信: 所有工作节点都与参数服务器通信以更新权重; 不能扩展到大量工作节点
- 如何在DNN培训中分散通信?
- AllReduce: 跨多个设备执行元素缩减

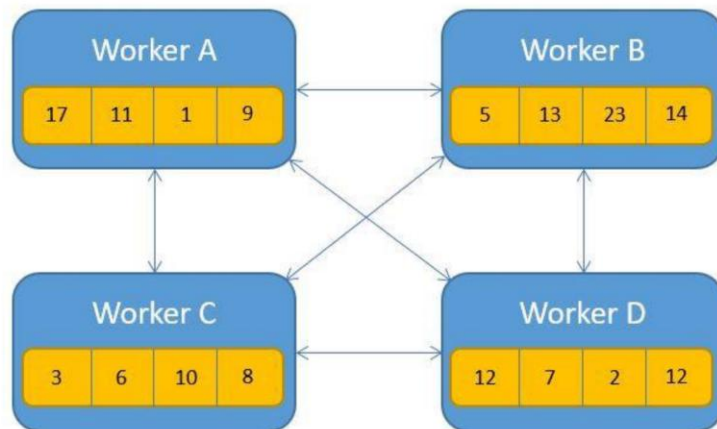


执行AllReduce的不同方式

- Naïve AllReduce
- Ring AllReduce
- Tree AllReduce
- Butterfly AllReduce

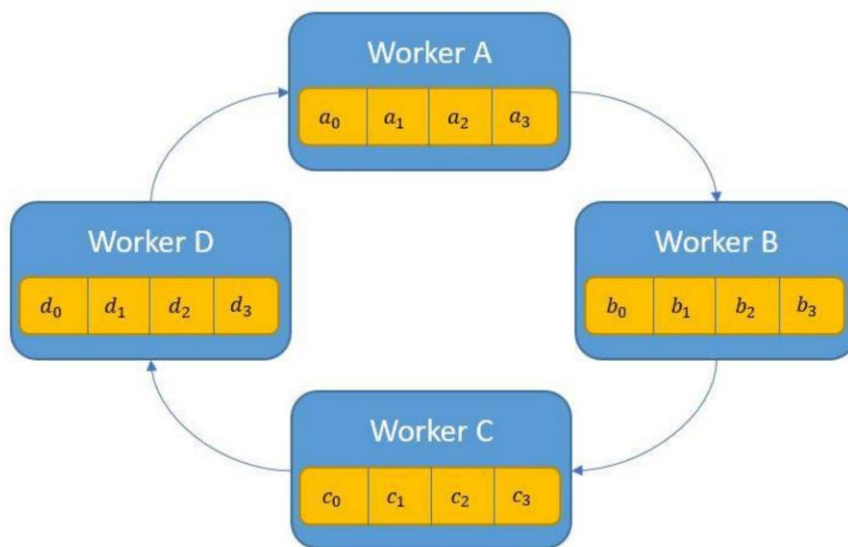
Naïve AllReduce

- 每个工作节点可以将其本地梯度发送给所有其他工作节点
- 若有N个工作节点,每个工作节点包含M个参数
- 总通信: $N*(N-1)*M$ 参数
- 问题:每个工作节点与所有其他工作节点通信;与参数服务器相同的可扩展性问题



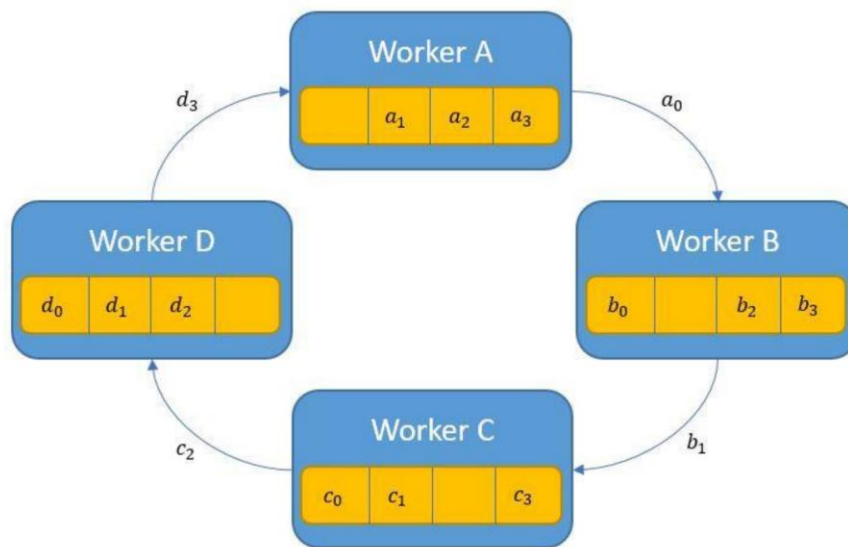
Ring AllReduce

- 构N个工作节点的环,将M个参数分为N个切片
- 第1步(聚合):每个worker向环上的下一个worker发送一个切片(M/N参数);
重复N次



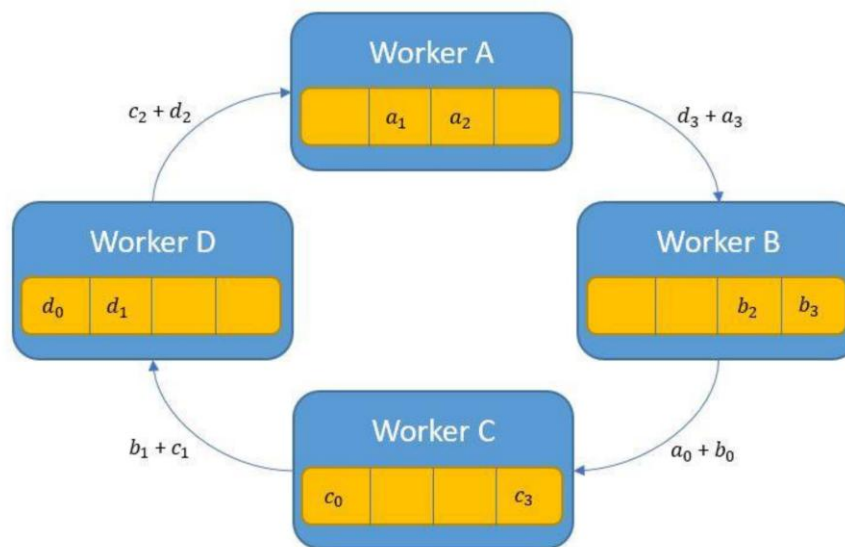
Ring AllReduce

- 构造N个工作节点的环,将M个参数分为N个切片
- 第1步(聚合):每个worker向环上的下一个worker发送一个切片(M/N参数);重复N次



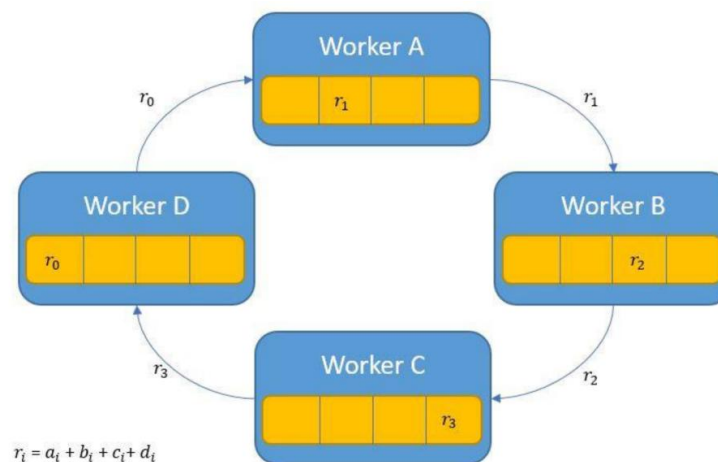
RingAllReduce

- 构N个工作节点的环,将M个参数分为N个切片
- 第1步(聚合):每个worker向环上的下一个worker发送一个切片(M/N参数);
重复N次



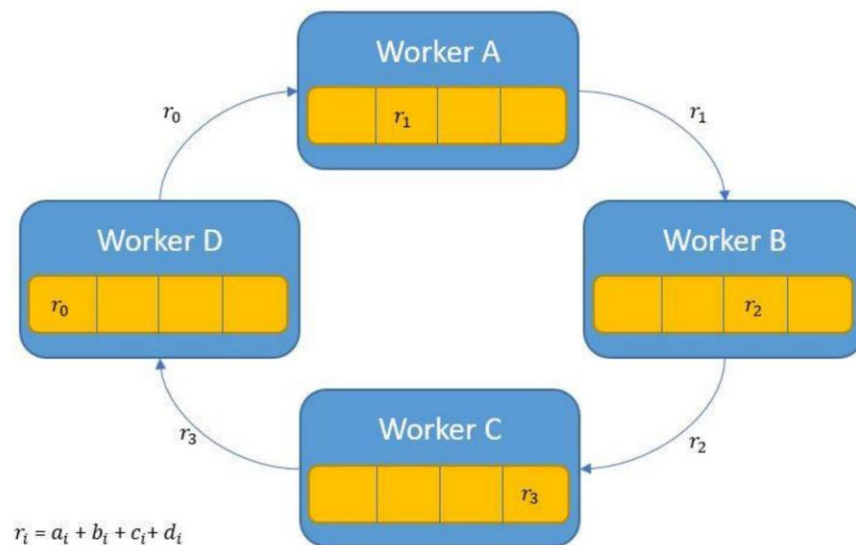
Ring AllReduce

- 构N个工作节点的环,将M个参数分为N个切片
- 第1步(聚合):每个worker向环上的下一个worker发送一个切片(M/N参数);
重复N次
- 步骤1之后,每个工作节点都有M/N参数的聚合版本



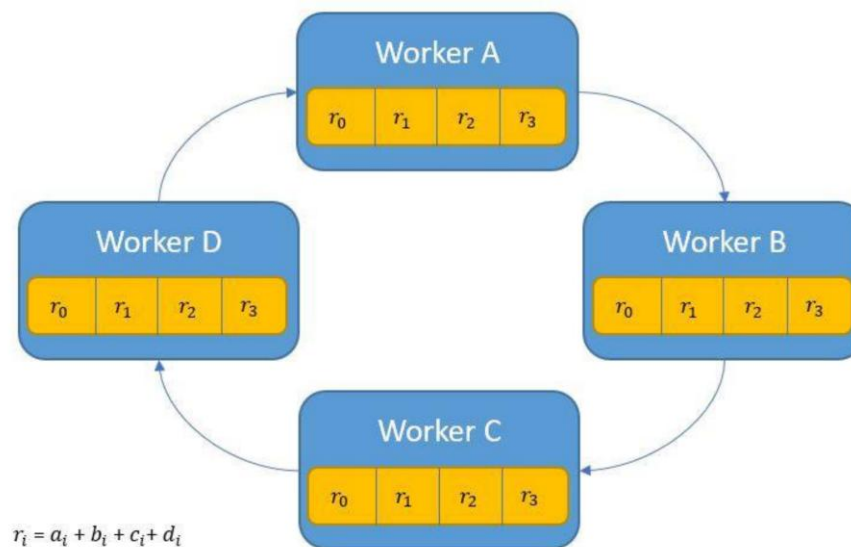
Ring AllReduce

- 构N个工作节点的环,将M个参数分为N个切片
- 第1步(聚合):每个worker向环上的下一个worker发送一个切片(M/N参数);重复N次
- 第2步(广播):每个工作节点将聚合参数的一片发送给下一个工作节点;重复N次



Ring AllReduce

- 构N个工作节点的环,将M个参数分为N个切片
- 第1步(聚合):每个worker向环上的下一个worker发送一个切片(M/N参数);重复N次
- 第2步(广播):每个工作节点将聚合参数的一片发送给下一个工作节点;重复N次

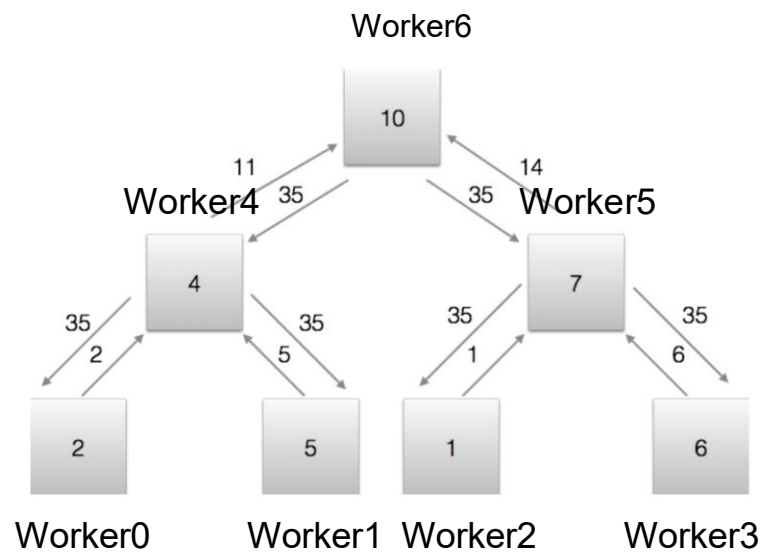


Ring AllReduce

- 构造N个工作节点的环,将M个参数分为N个切片
- 第1步(聚合):每个worker向环上的下一个worker发送一个切片(M/N 参数);重复N次
- 第2步(广播):每个工作节点将聚合参数的一片发送给下一个工作节点;重复N次
- 总通信: $2 * M * N$ 参数
- 聚合: $M * N$ 参数
- 广播: $M * N$ 参数

Tree AllReduce

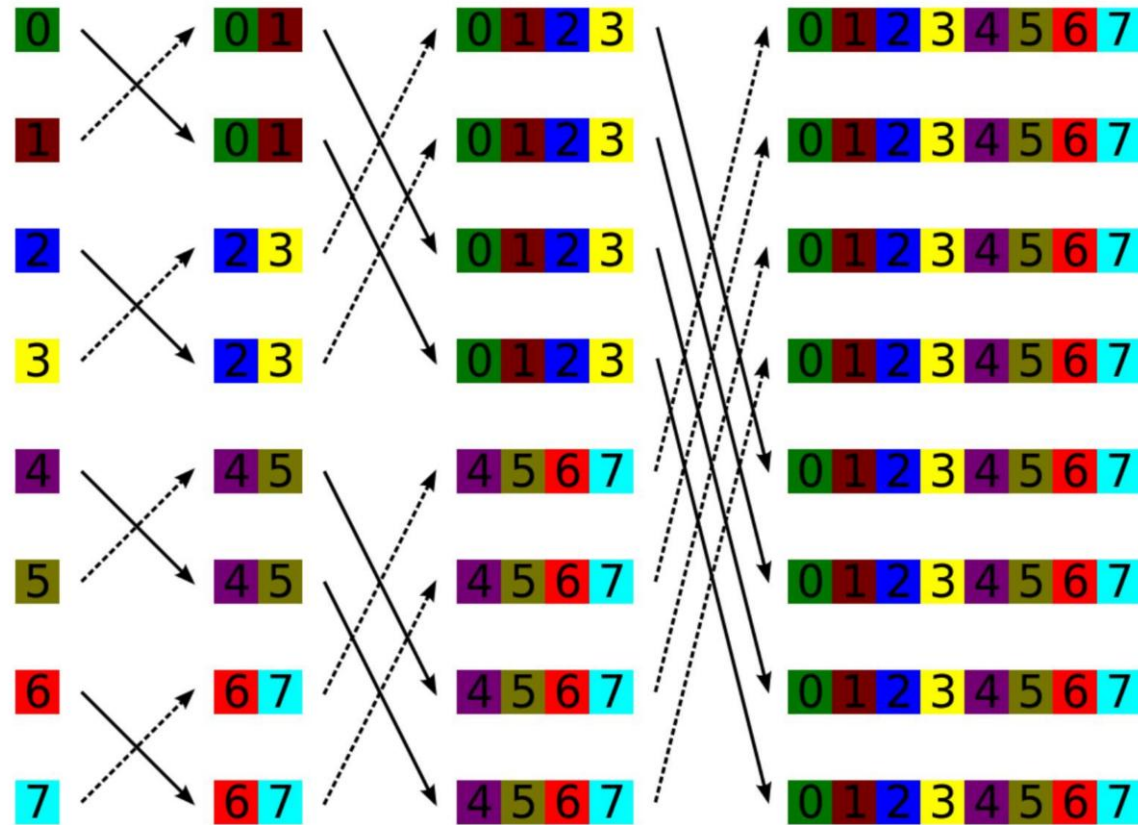
- Step1(构建一个包含N个工作节点的树结构):每个工作程序将M参数发送给其父级;重复 $\log(N)$ 次
- Step2(广播):每个工作节点将M参数发送给其子级;重复 $\log(N)$ 次



TreeAllReduce

- Step1(构建一个包含N个工作节点的树结构):每个工作程序将M参数发送给其父级;
重复 $\log(N)$ 次
- Step2(广播):每个工作节点将M参数发送给其子级;重复 $\log(N)$ 次
- 总通信: $2 * N * M$ 参数
- 聚合: $M * N$ 参数
- 广播: $M * N$ 参数

Butterfly Network

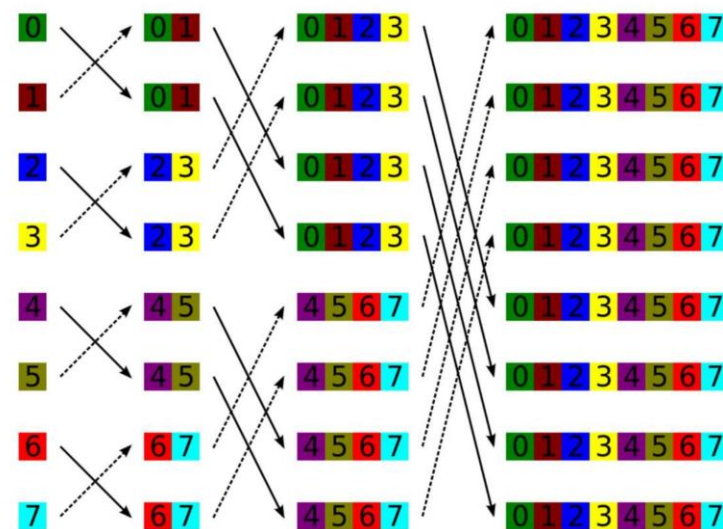


Butterfly AllReduce

• 重复 $\log(N)$ 次:

1. 每个工作节点将 M 个参数发送到蝴蝶网络中的目标节点
2. 每个工作节点在本地聚合梯度

• 总通信: $N * M * \log(N)$ 参数



不同的AllReduce方法

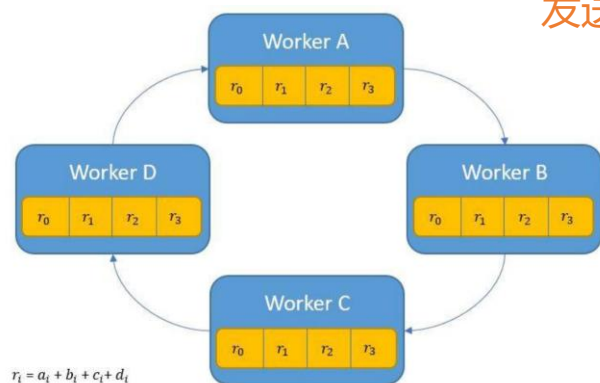
	Parameter Server	Naïve AllReduce	Ring AllReduce	Tree AllReduce	Butterfly AllReduce
Overall communication	$2 \times N \times M$	$N^2 \times M$	$2 \times N \times M$	$2 \times N \times M$	$N \times M \times \log N$

问题: Ring AllReduce 比 Tree AllReduce 和参数服务器更高效、更可扩展, 为什么?

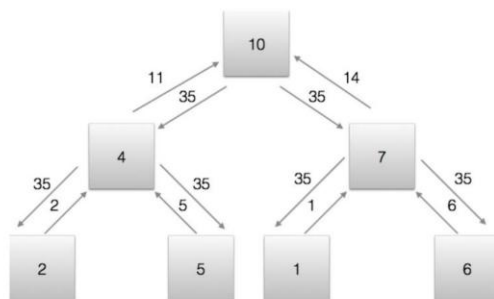
AllReduce实现对比

RingAllReduce:

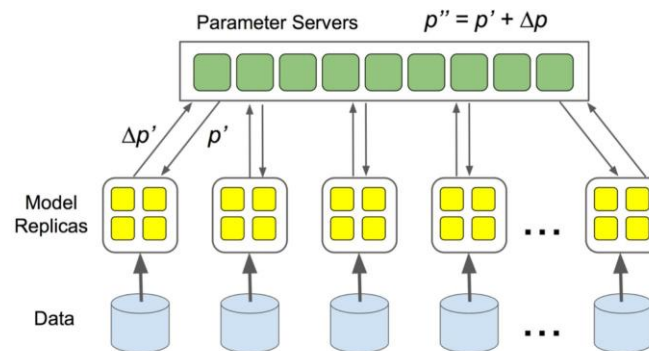
- 最佳延迟
- 工作节点之间的工作量平衡
- 由于每个工作节点都更具可扩展性
发送 $2 \cdot M$ 参数(与工作节点数无关)



每个Worker发送 M/N 参数
迭代;重复 $2 \cdot N$ 次迭代
延迟: $M/N \cdot (2 \cdot N) / \text{带宽}$



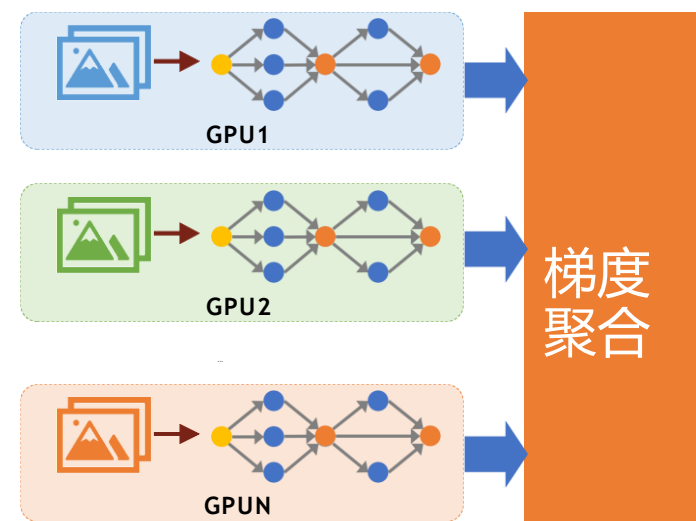
每个工作节点每次迭代发送 M 个参数;
重复 $2 \cdot \log(N)$ 迭代延迟: $M \cdot 2 \cdot \log(N) / \text{带宽}$



所有工作节点向参数服务器发送 M
个参数,从服务器接收 M 个参数
延迟: $M \cdot N / \text{带宽}$

数据并行问题

- 每个GPU保存整个模型的副本
- 不能训练超过GPU设备内存的大型模型



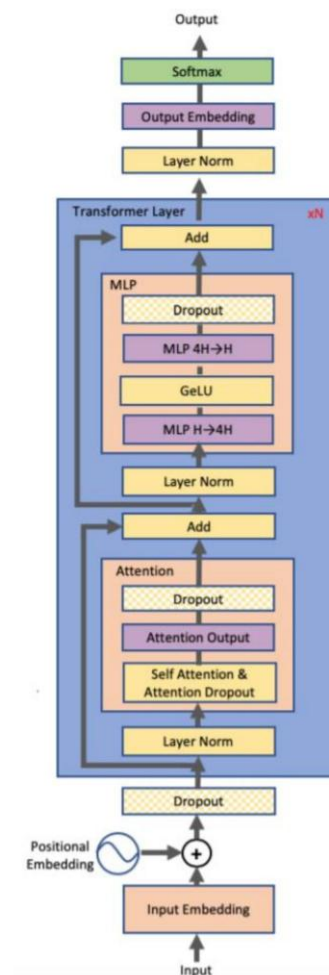
目录

Contents

- 1 分布式深度学习训练
- 2 数据并行与梯度聚合
- 3 大模型训练的内存挑战**
- 4 ZeRO与全分片数据并行

大模型训练挑战

	Bert-Large	GPT- 2	Turing17.2NLG	GPT-3
参数	0.32B	1.5B	17.2B	175B
层数	24	48	78	96
隐藏维度	1024	1600	4256	12288
相对计算	1x	4.7x	54x	547x
内存占用	5.12GB	24GB	275GB	2800GB

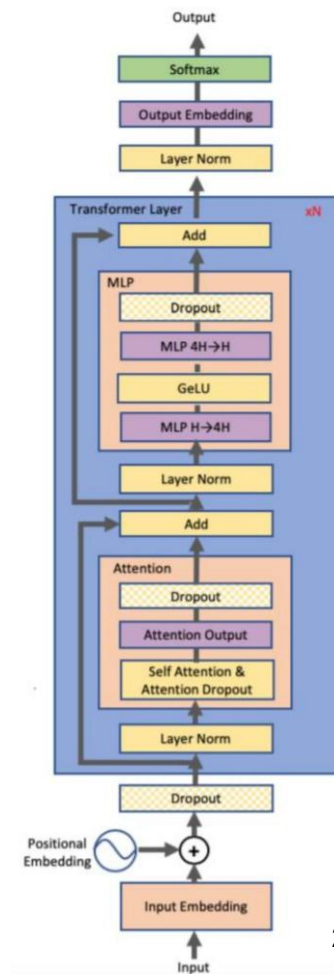


大模型训练挑战

	Bert-Large	GPT- 2	Turing17.2NLG	GPT-3
参数	0.32B	1.5B	17.2B	175B
层数	24	48	78	96
隐藏维度	1024	1600	4256	12288
相对计算	1x	4.7x	54x	547x
内存占用	5.12GB	24GB	275GB	2800GB

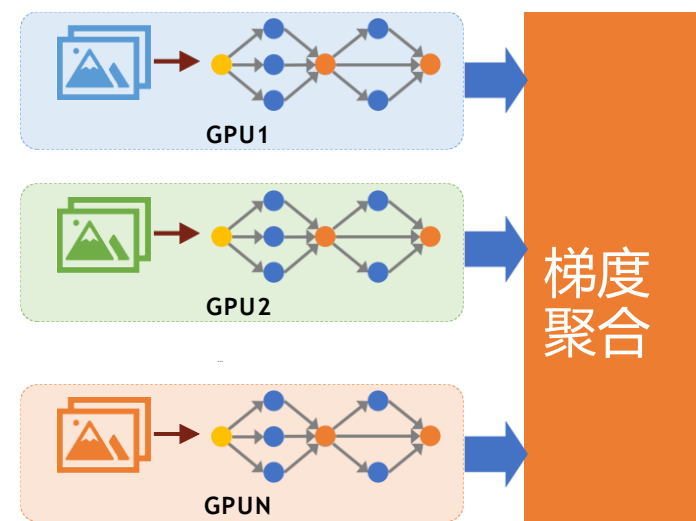
NVIDIA V100 GPU 内存容量: 16G/32G
 NVIDIA A100 GPU 内存容量: 40G/80G

Out of Memory



ZeRO:零冗余优化器

- 消除数据中的数据冗余
- 并行训练
- 大模型数据并行训练的广泛使用的技术



随机梯度下降

For $t=1$ to T

$\frac{1}{b} \sum_{i=1}^b \nabla \left(\text{loss}(f_w(x_i, y_i)) \right)$ // $\Delta w = \eta \times$ 计算导数和更新

$w = w - \Delta w$ // 应用更新

End

自适应学习(Adam)

For $t=1$ to T

$$g = \frac{1}{b} \sum_{i=1}^b \nabla \left(\text{loss}(f_w(x_i, y_i)) \right)$$

$$\Delta w = \text{adam}(g)$$

$w \leftarrow w + \Delta w$ // apply update

End

$$\begin{aligned} \nu_t &= \beta_1 * \nu_{t-1} + (1 - \beta_1) * g_t \\ s_t &= \beta_2 * s_{t-1} + (1 - \beta_2) * g_t^2 \\ \Delta \omega_t &= -\eta \frac{\nu_t}{\sqrt{s_t + \epsilon}} * g_t \end{aligned}$$

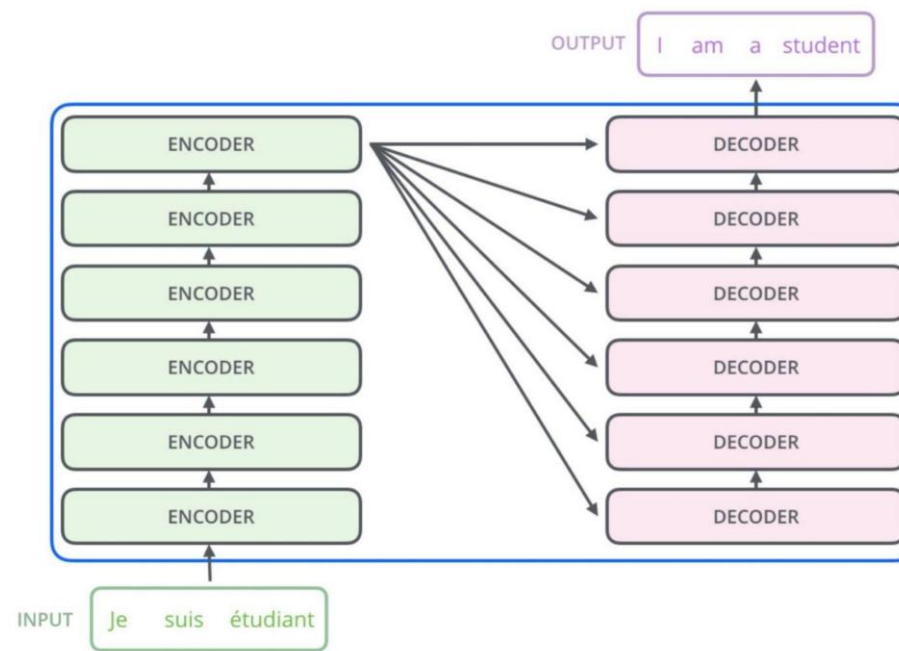
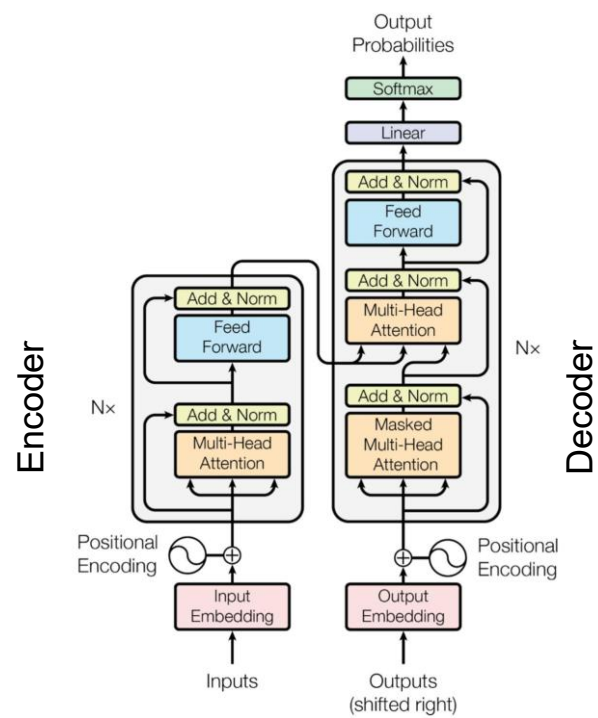
g_t : Gradient at time t along ω^j

ν_t : Exponential Average of gradients along ω_j

s_t : Exponential Average of squares of gradients along ω_j

β_1, β_2 : Hyperparameters

Transformer用于大模型

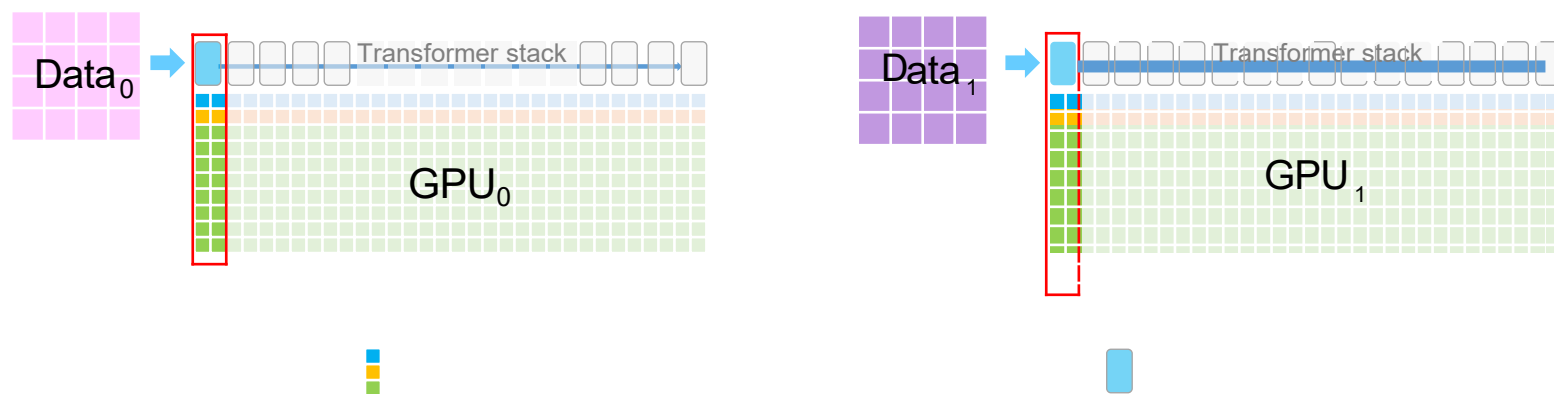


理解内存消耗



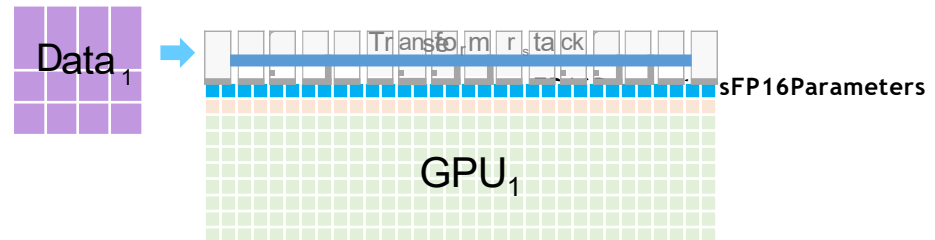
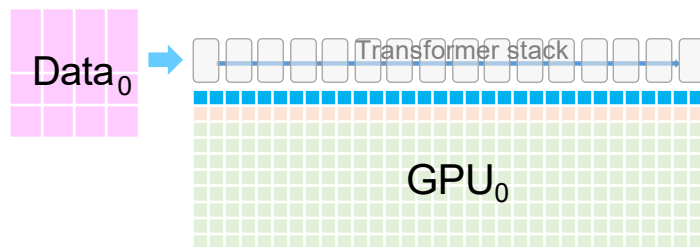
一个16层transformer=1 layer

理解内存消耗



每个单元格代表其对应transformer使用的GPU内存

理解内存消耗



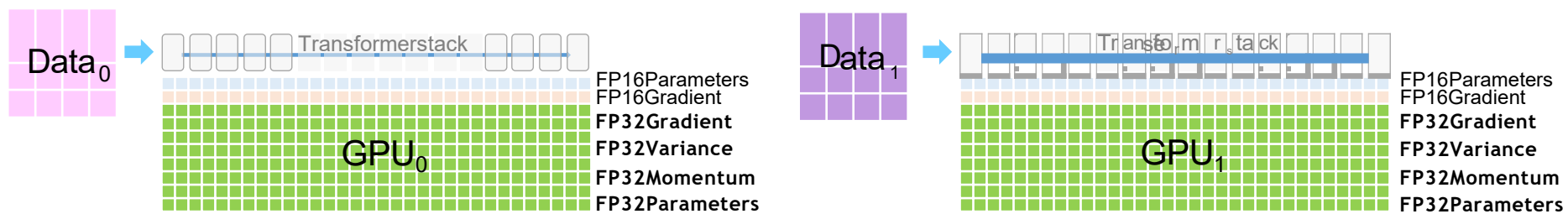
•FP16参数

理解内存消耗



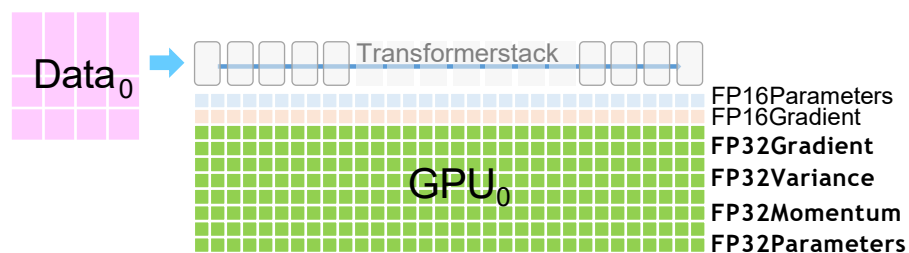
- FP16参数
- FP16梯度

理解内存消耗

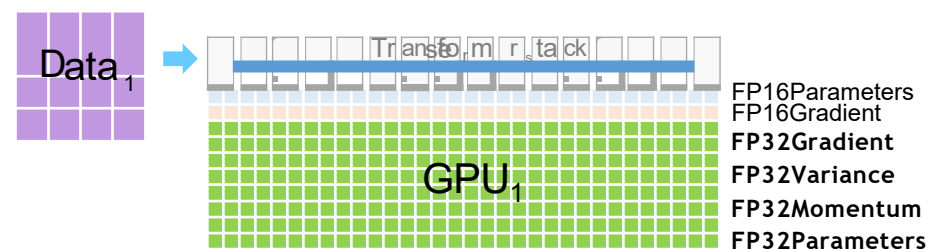


- FP16参数
- FP16导数
- FP32优化器状态
 - 梯度、方差、动量、参数

理解内存消耗



- FP16参数:2Mbytes
- FP16导数:2Mbytes
- FP32优化器状态:16Mbytes
 - 梯度、方差、动量,M=模型中的参数



示例1B参数模型->20GB/GPU

参数内存消耗不包括:

- 输入批次+激活

目录

Contents

- 1 分布式深度学习训练
- 2 数据并行与梯度聚合
- 3 大模型训练的内存挑战
- 4 ZeRO与全分片数据并行**

ZeRO消除跨数据并行进程的冗余

•Stage1:分区优化器状态

核心功能：将优化器状态（如Adam的动量和方差）均匀分片到不同GPU上，每个GPU只维护 $1/N$ 的优化器状态

显存节省：将优化器状态的显存占用从 O 减少到 O/N （ N 为GPU数量）

适用场景：当优化器状态是主要显存瓶颈时（如使用Adam优化器训练中等规模模型）

通信开销：与传统数据并行基本相当，主要在参数更新时需要All-Gather操作

•Stage2:分区梯度

核心功能：在Stage 1基础上，进一步将梯度分片，每个GPU只保留与自己优化器状态分片对应的梯度

显存节省：将梯度显存占用从 P 减少到 P/N （ P 为模型参数量）

适用场景：当模型规模增大，梯度成为新的显存瓶颈时

通信开销：需要Reduce-Scatter和All-Gather操作，通信量约为 $2P$ （与Stage 1相当）

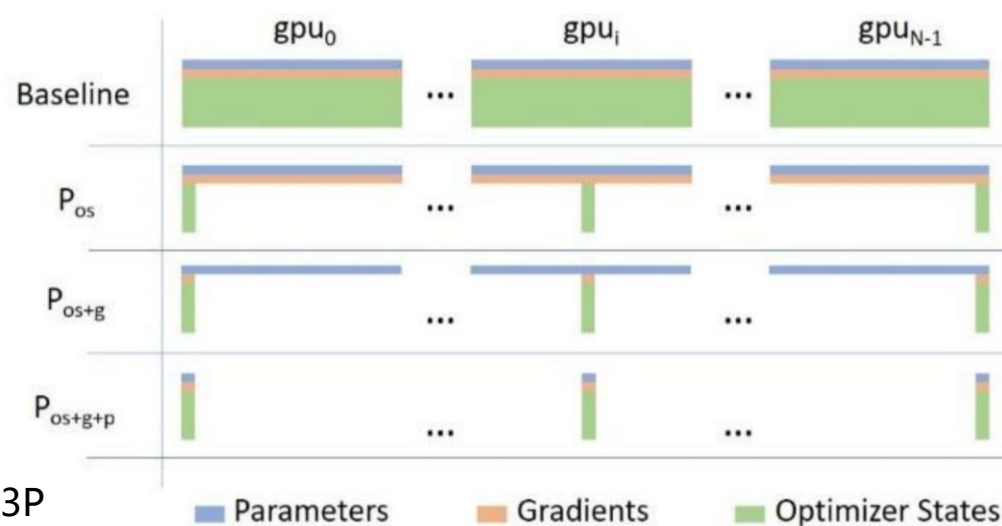
•Stage3:分区参数

核心功能：在Stage 2基础上，将模型参数本身也分片，每个GPU只持有部分参数

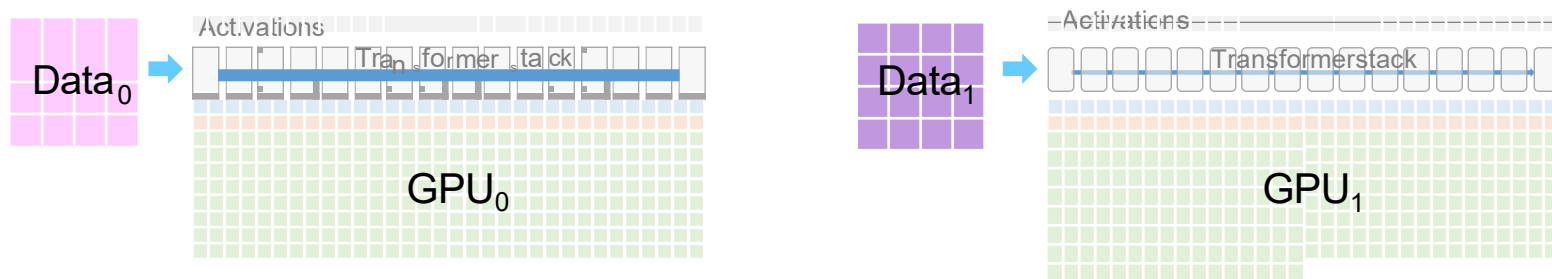
显存节省：将参数显存占用从 P 减少到 P/N ，实现显存占用与GPU数量的线性关系

适用场景：超大规模模型（如百亿、千亿参数）训练，单卡无法容纳完整模型时

通信开销：显著增加，需要在正向/反向传播过程中动态获取其他参数分片，通信量约为 $3P$



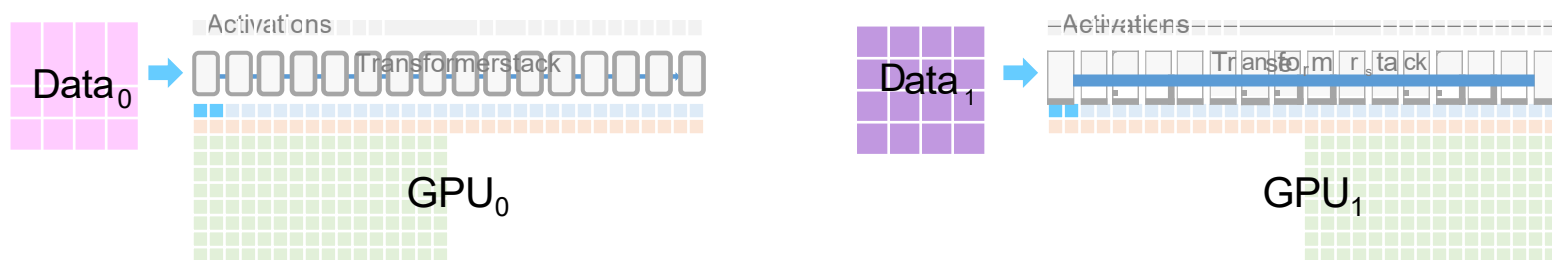
ZeRO Stage1:分区优化器状态



•ZeROStage1

Parameters Gradients Optimizer States

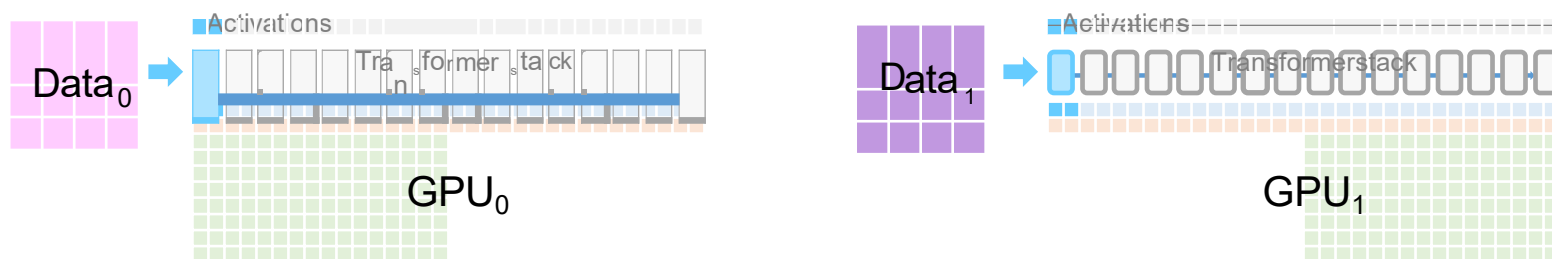
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态

Parameters Gradients Optimizer States

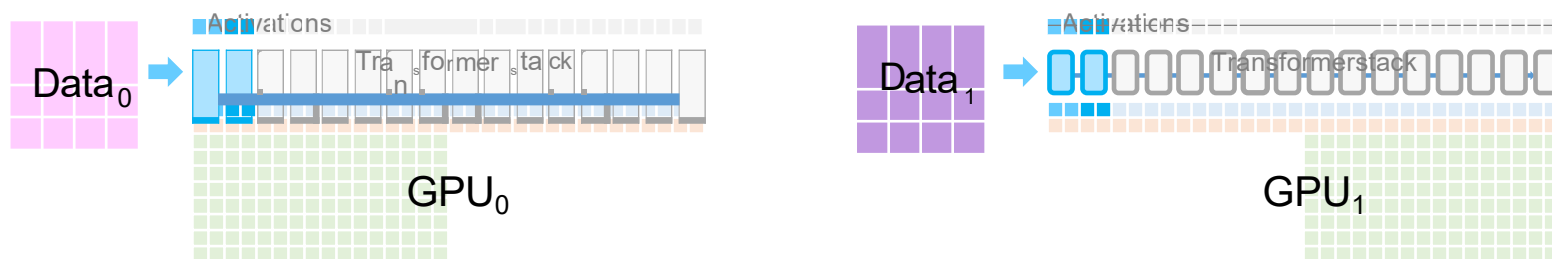
ZeRO Stage1:分区优化器状态



- ZeRO Stage 1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播

Parameters Gradients Optimizer States

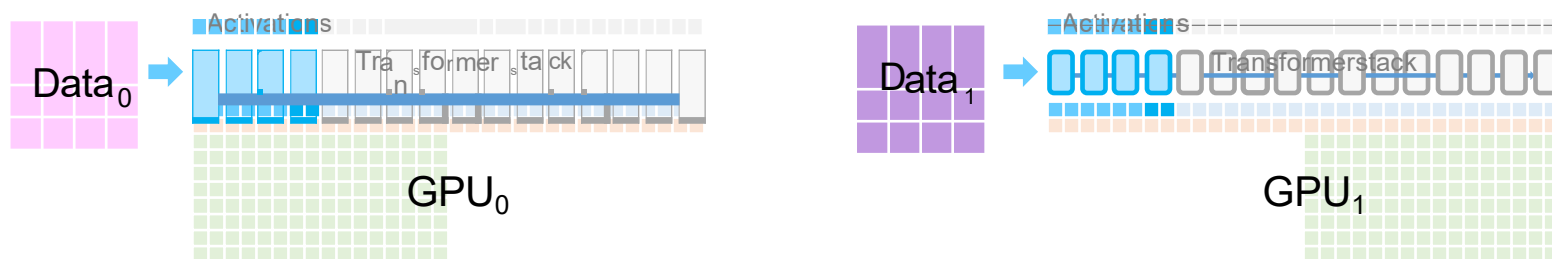
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播

Parameters Gradients Optimizer States

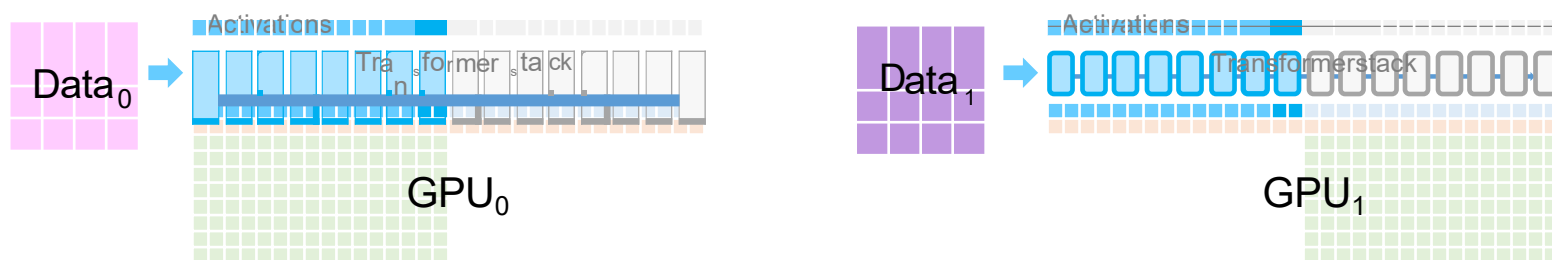
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播

Parameters Gradients Optimizer States

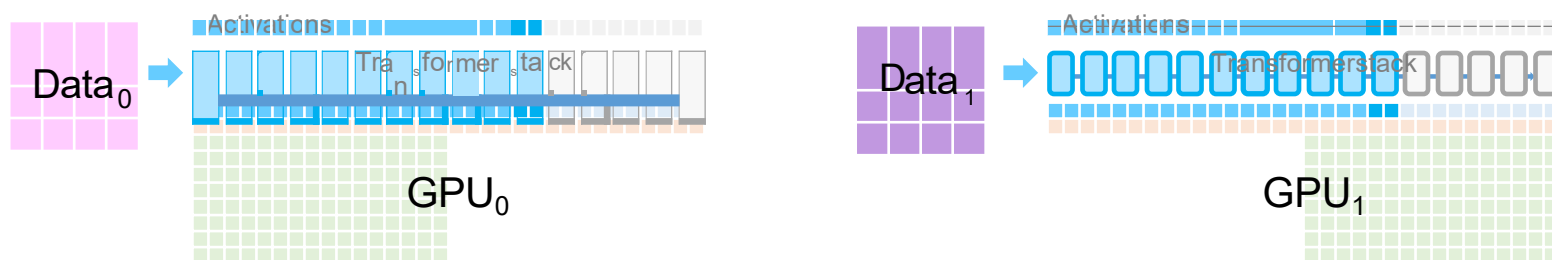
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播

Parameters Gradients Optimizer States

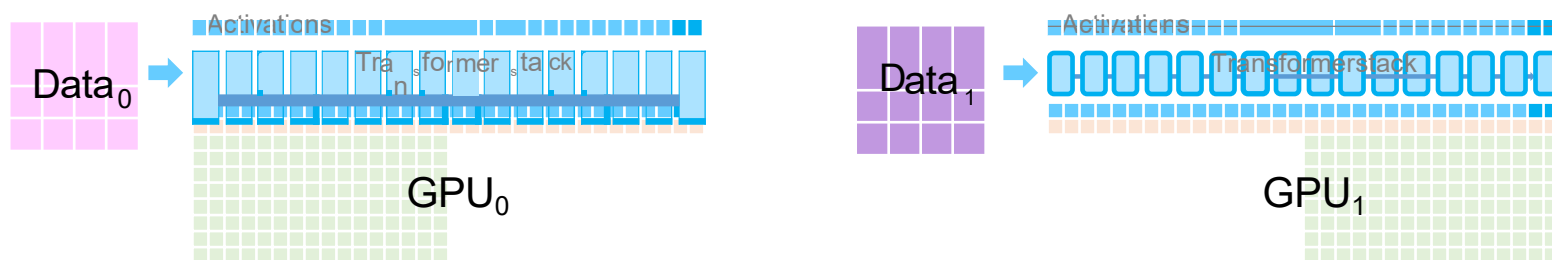
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播

Parameters Gradients Optimizer States

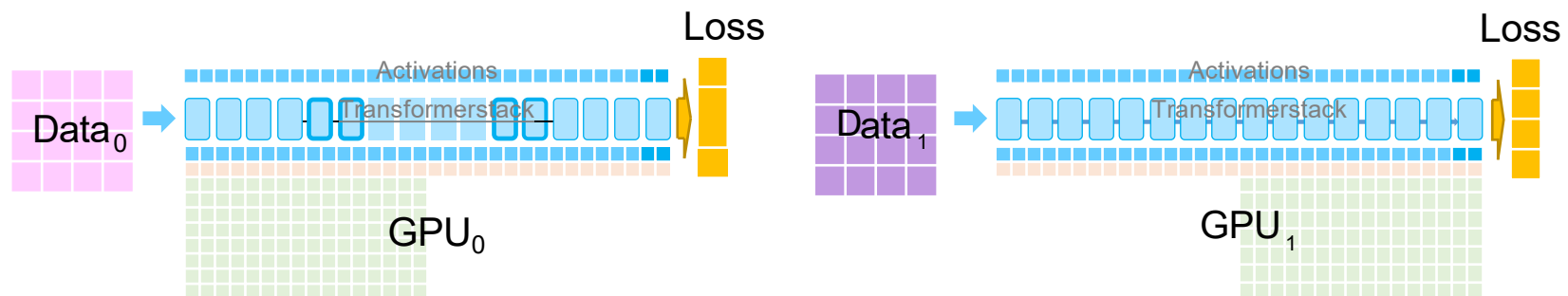
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播

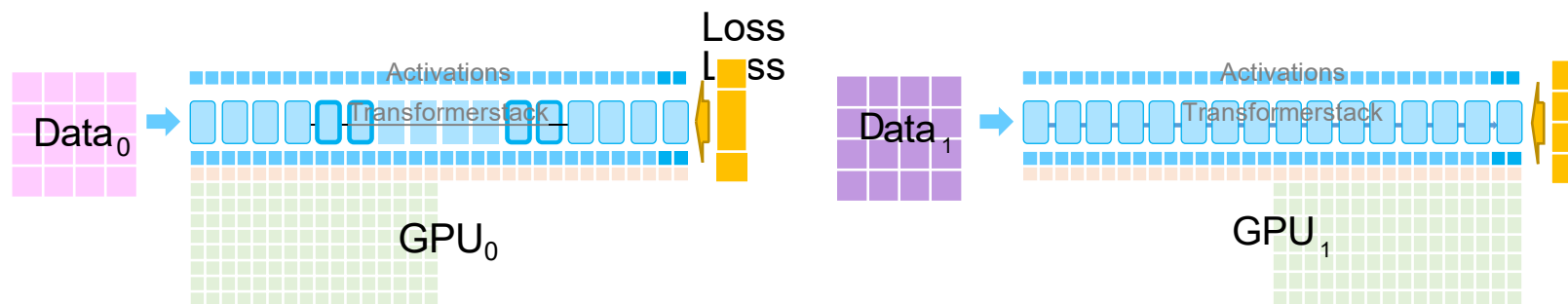
■ Parameters ■ Gradients ■ Optimizer States

ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播

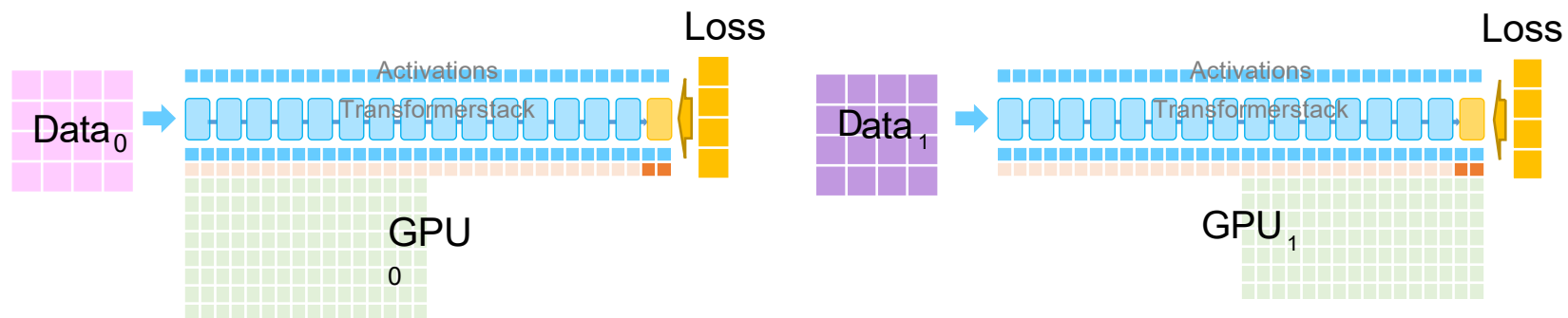
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播以生成FP16梯度

Parameters Gradients Optimizer States

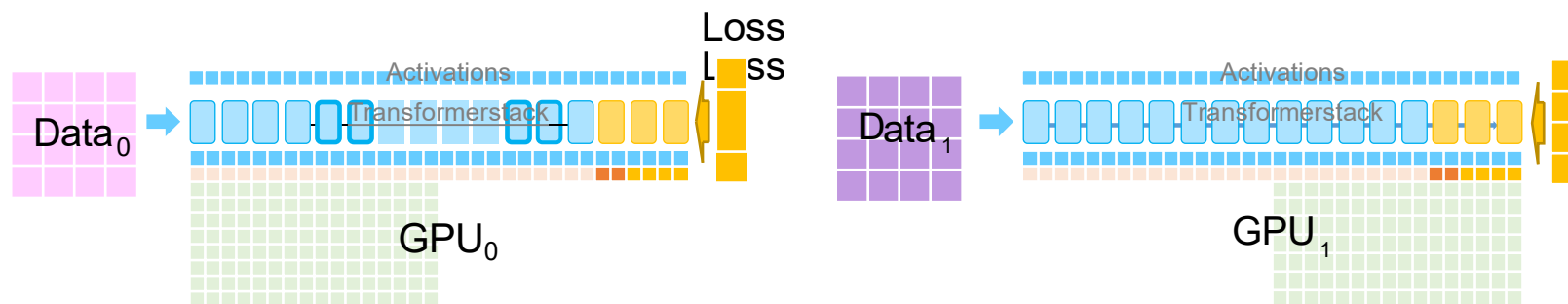
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播以生成FP16梯度

Parameters Gradients Optimizer States

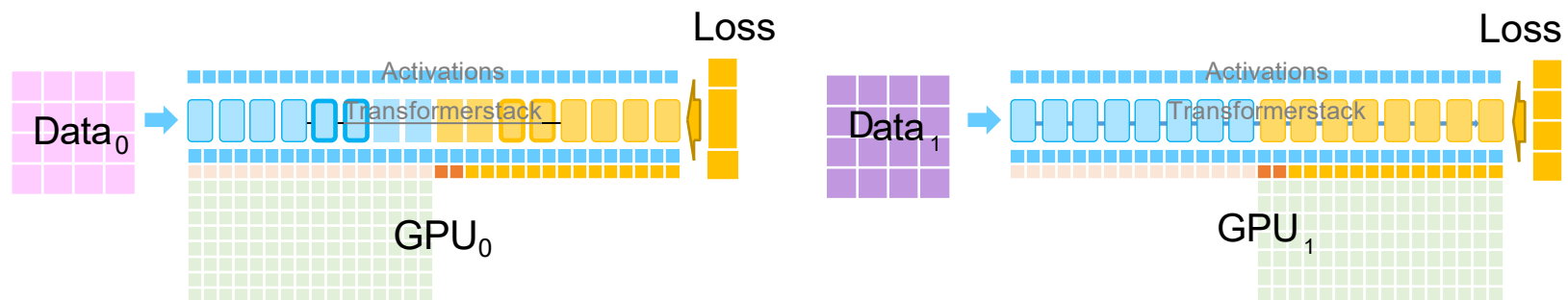
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播以生成FP16梯度

Parameters Gradients Optimizer States

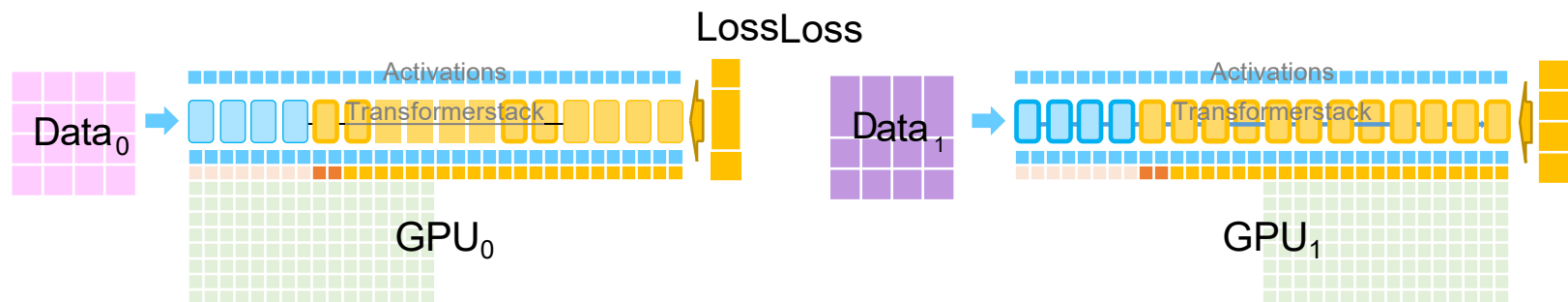
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播以生成FP16梯度

■ Parameters ■ Gradients ■ Optimizer States

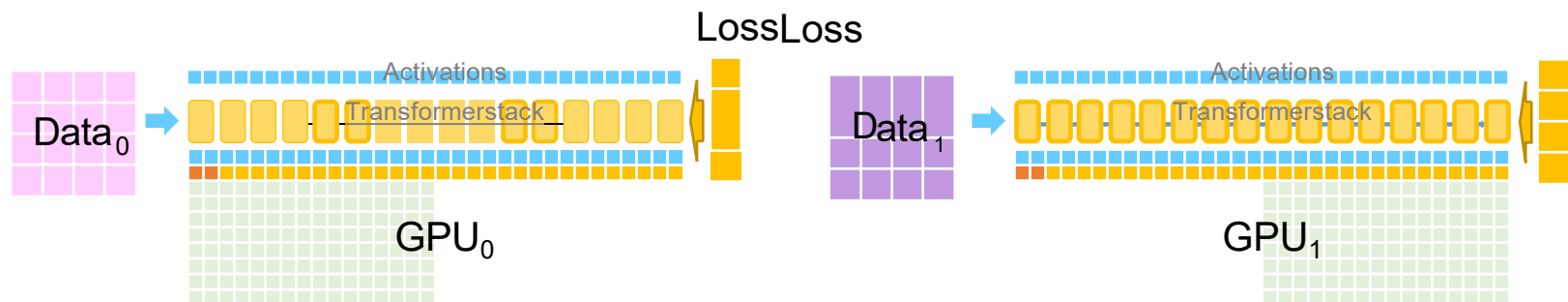
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播以生成FP16梯度

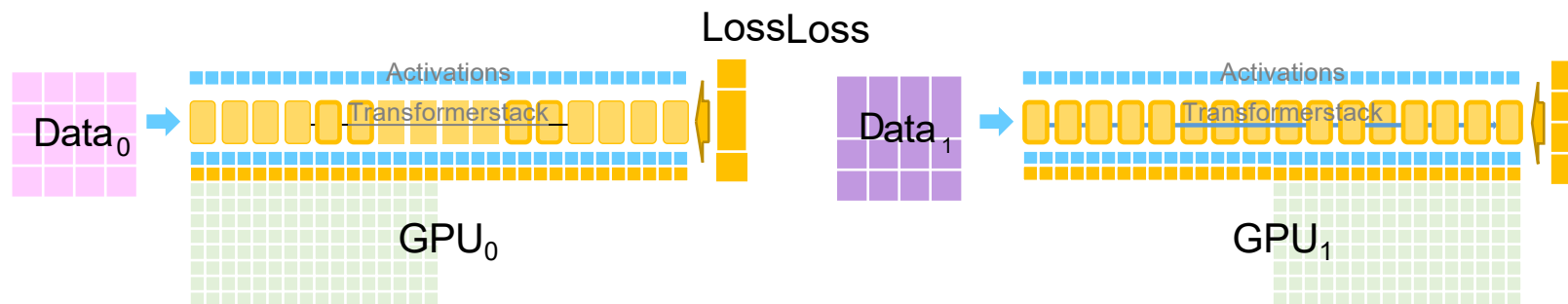
Parameters Gradients Optimizer States

ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播以生成FP16梯度

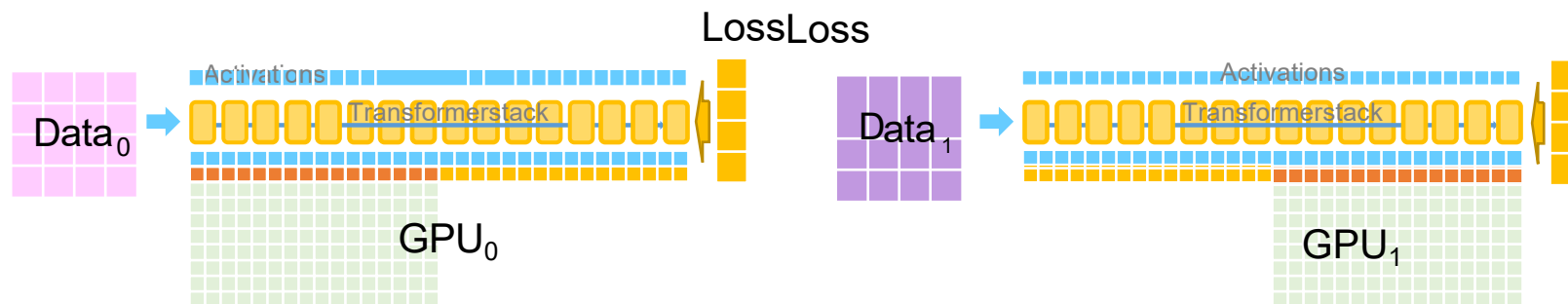
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度,AllReduce生成平均值

Parameters Gradients Optimizer States

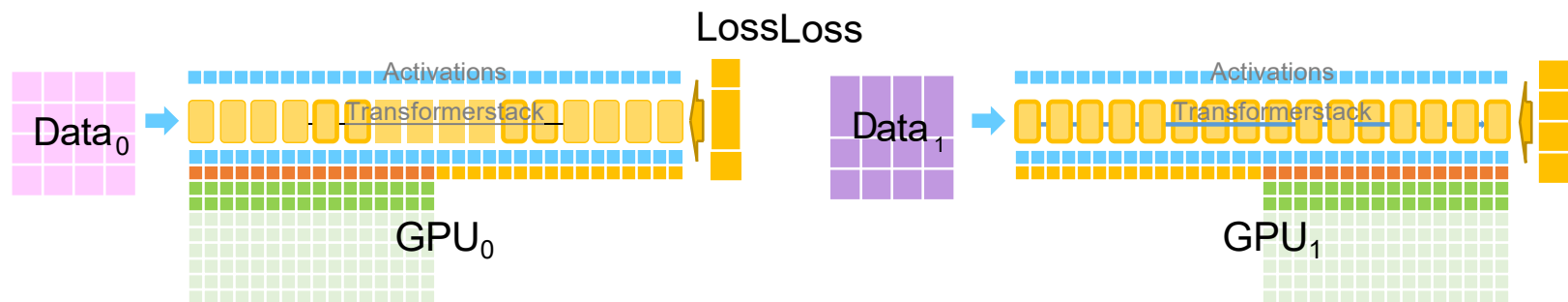
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度,AllReduce生成平均值

Parameters Gradients Optimizer States

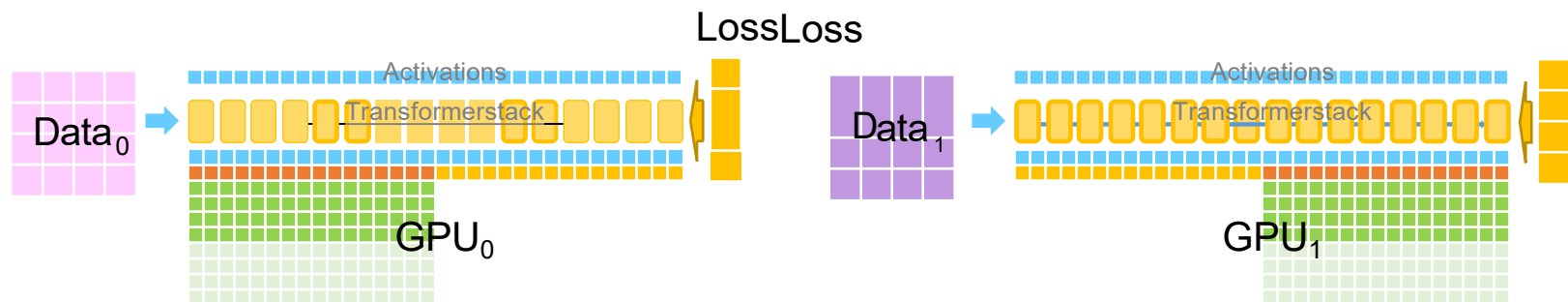
ZeRO Stage1:分区优化器状态



- ZeROStage1
- 跨GPU分区优化器状态
- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度,AllReduce生成平均值
- 使用ADAM优化器更新FP32权重

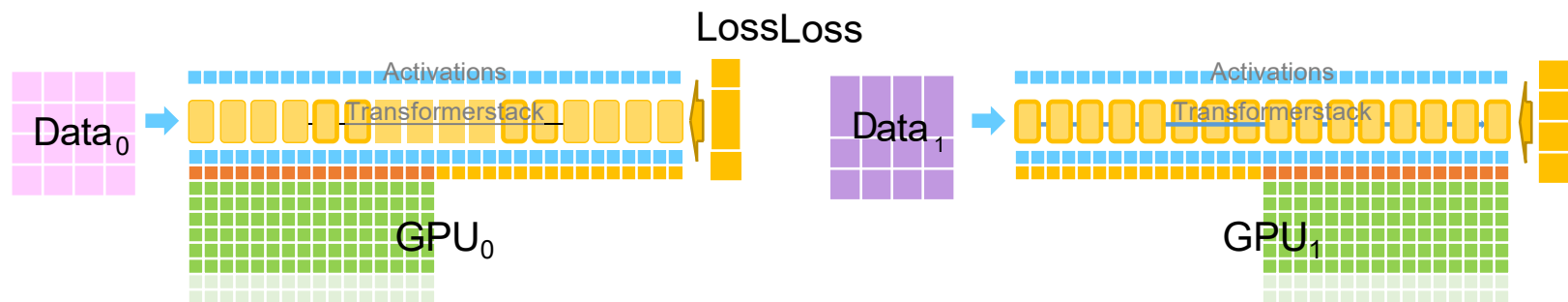
■ Parameters ■ Gradients ■ Optimizer States

ZeRO Stage1:分区优化器状态



- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度, AllReduce生成平均值
- 使用ADAM优化器更新FP32权重

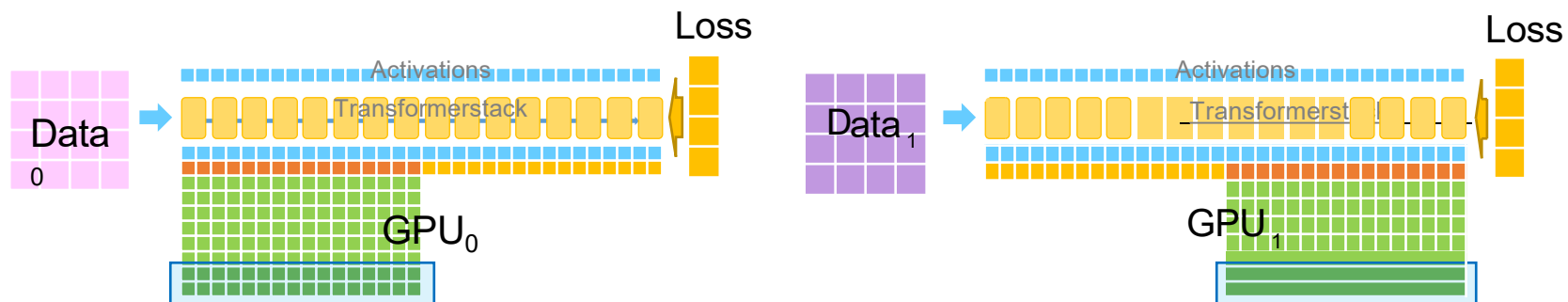
ZeRO Stage1:分区优化器状态



- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度, AllReduce生成平均值
- 使用ADAM优化器更新FP32权重

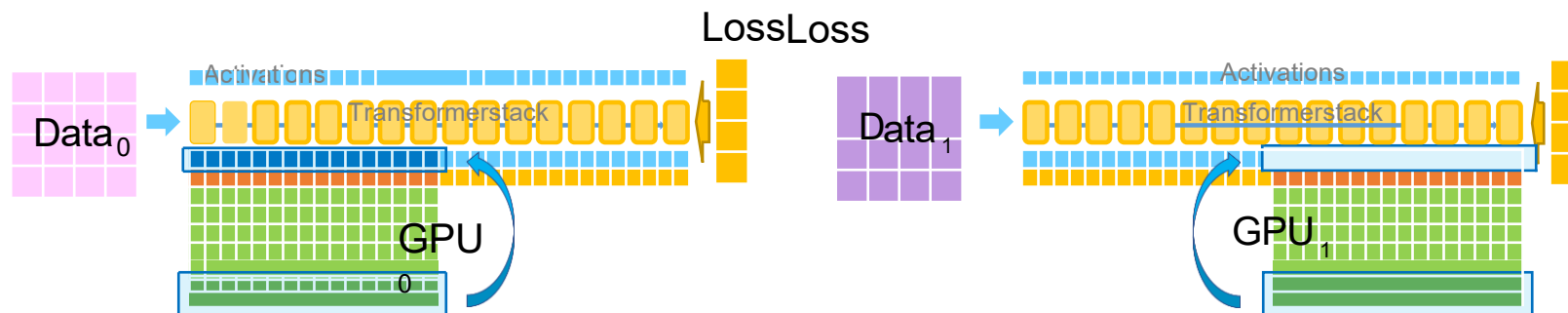
Parameters Gradients Optimizer States

ZeRO Stage1:分区优化器状态



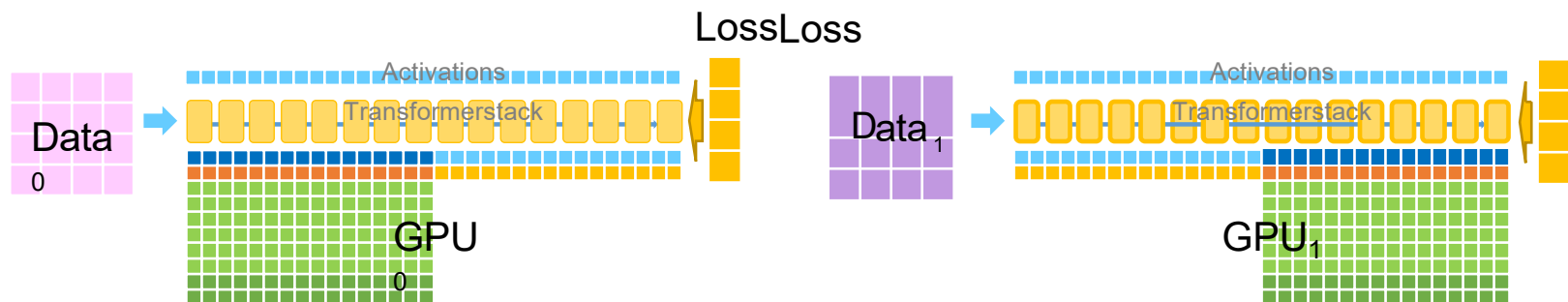
- 在Transformer各个模块 (Block) 中执行前向传播
- 向后传播生成FP16梯度, AllReduce生成平均值
- 使用ADAM优化器更新FP32权重

ZeRO Stage1:分区优化器状态



- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度, AllReduce生成平均值
- 使用ADAM优化器更新FP32权重
- 更新FP16权重

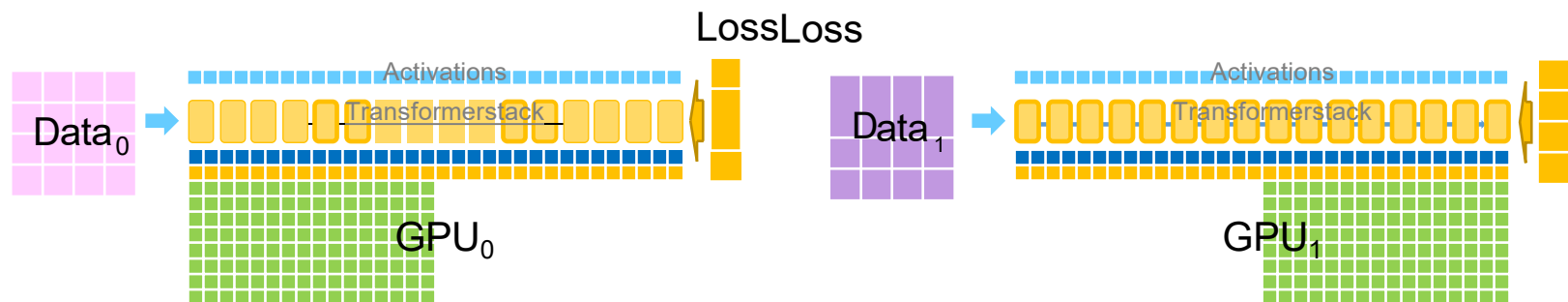
ZeRO Stage1:分区优化器状态



- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度, AllReduce生成平均值
- 使用ADAM优化器更新FP32权重
- 更新FP16权重
- 全部收集FP16权重以完成迭代

Parameters Gradients Optimizer States

ZeRO Stage1:分区优化器状态

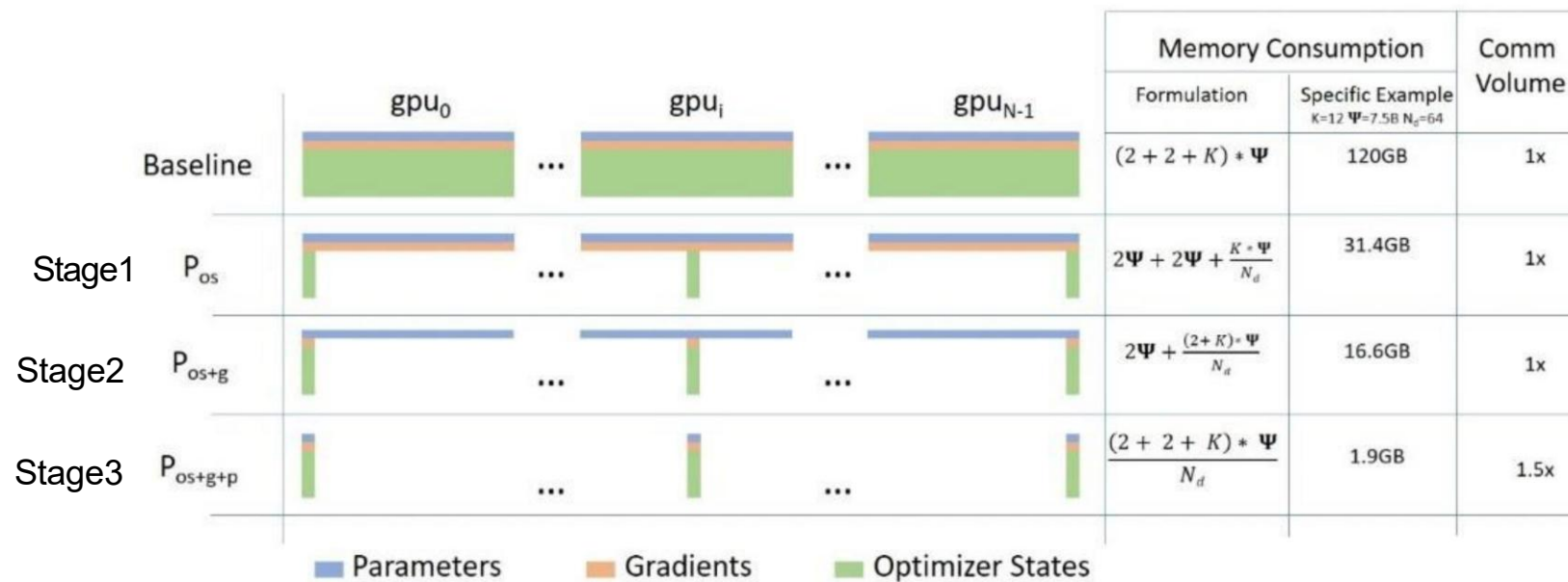


- 在Transformer各个模块（Block）中执行前向传播
- 向后传播生成FP16梯度, AllReduce生成平均值
- 使用ADAM优化器更新FP32权重
- 更新FP16权重
- 全部收集FP16权重以完成迭代

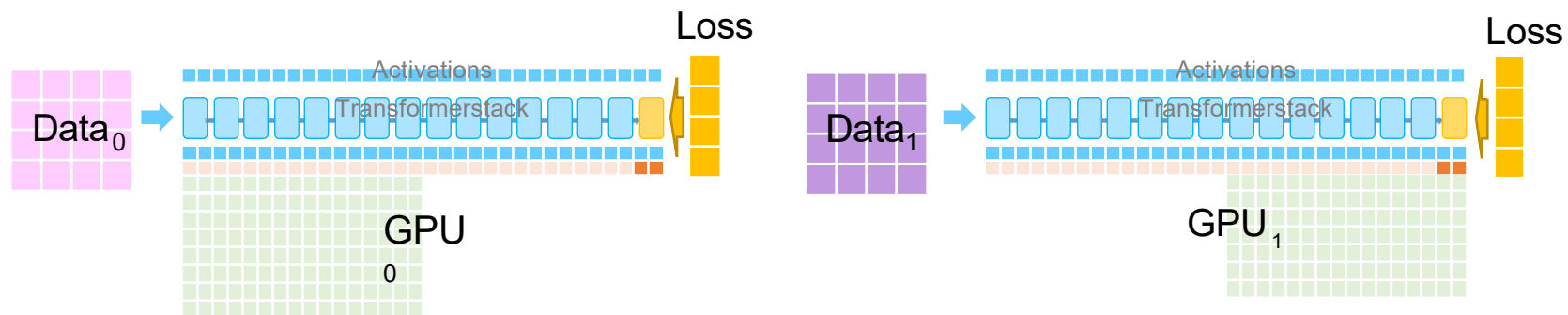
Parameters Gradients Optimizer States

ZeRO: Zero Redundancy Optimizer

- 渐进式内存节省和通信量
- NLR17.2B模型的训练/微调是基于Stage1分布式优化和Megatron框架实现的



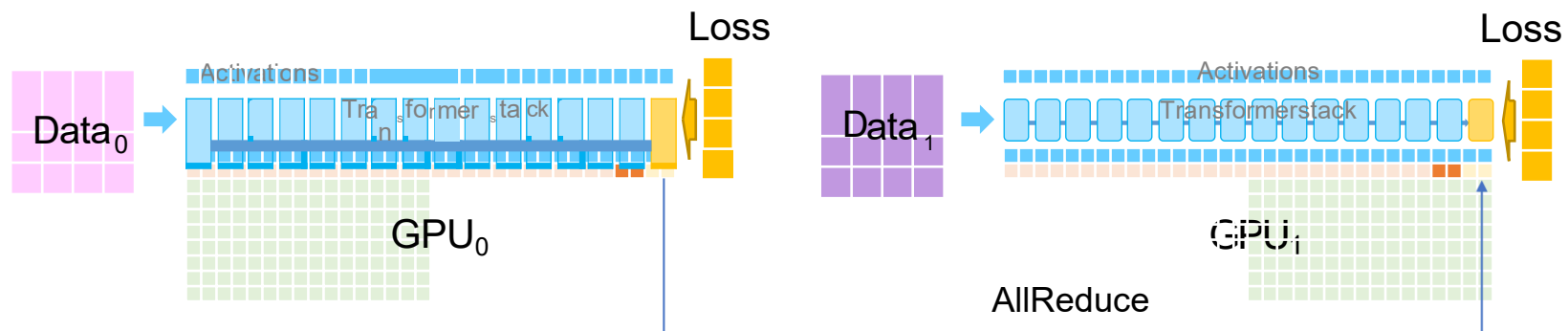
ZeRO Stage2:梯度分区



- 跨GPU划分梯度
- 前向过程与第一阶段相同

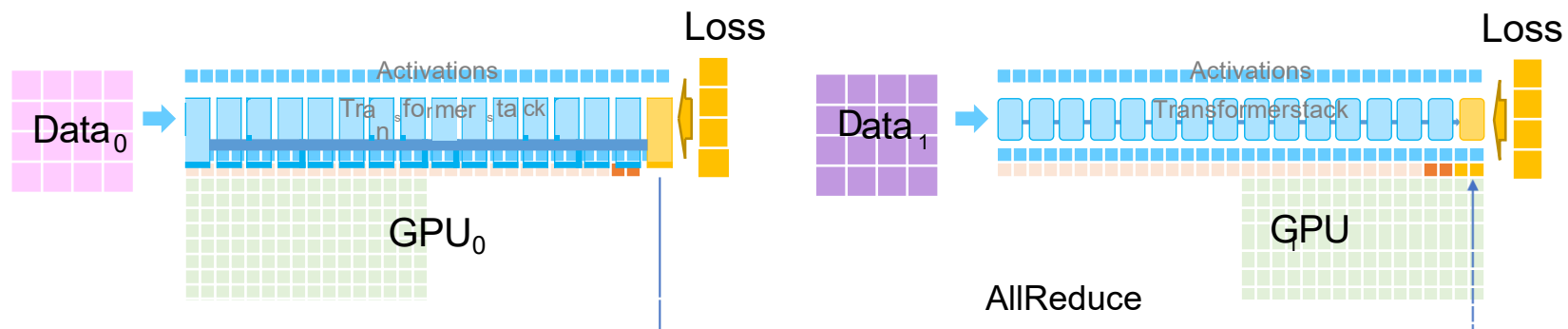
Parameters Gradients Optimizer States

ZeRO Stage2:梯度分区



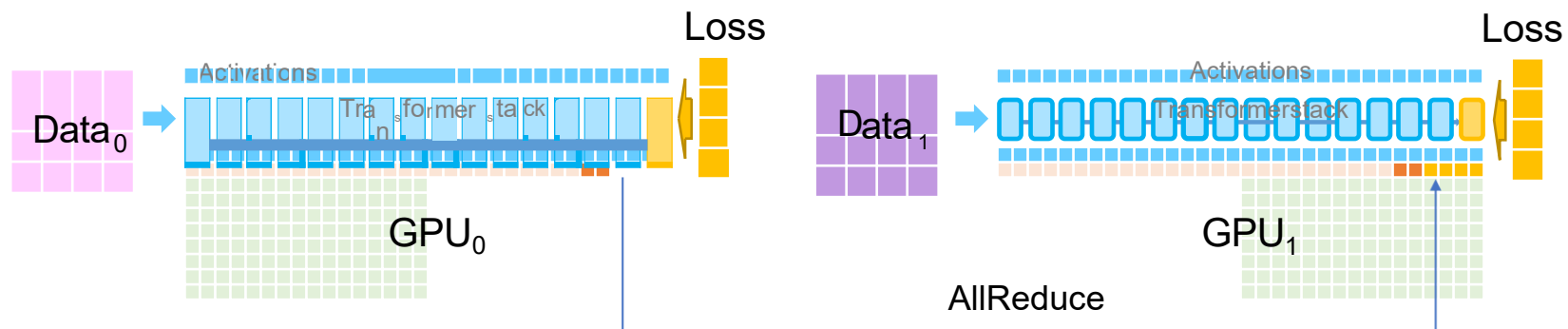
- 跨GPU划分梯度
- 在每层反向传播后立即执行AllReduce

ZeRO Stage2:梯度分区



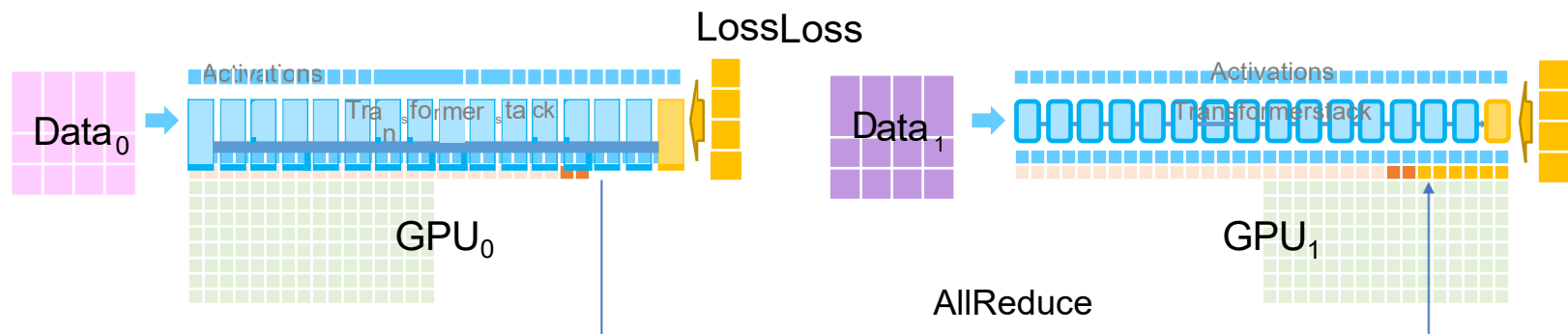
- 跨GPU划分梯度
- AllReduce之后只有一个GPU保留梯度

ZeRO Stage2:梯度分区



- 跨GPU划分梯度
- 减少负责更新参数的GPU上的梯度

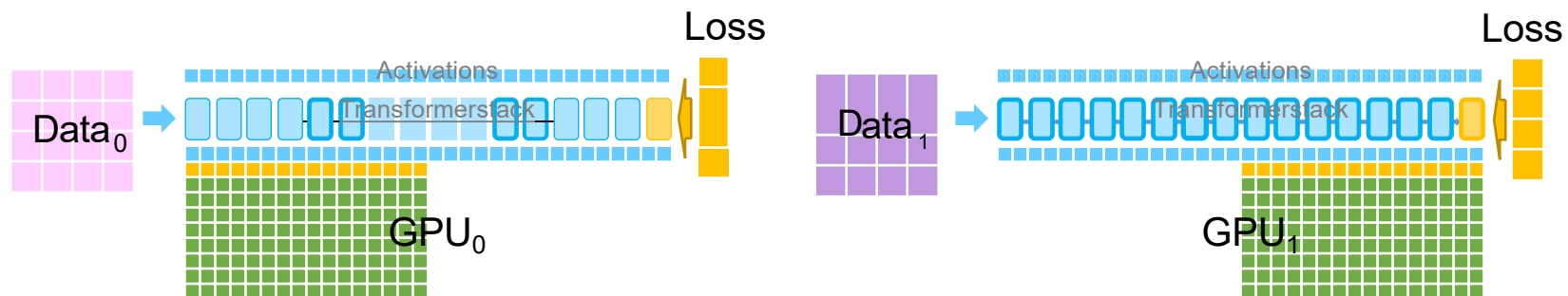
ZeRO Stage2:梯度分区



- 跨GPU划分梯度
- 减少负责更新参数的GPU上的梯度

Parameters Gradients Optimizer States

ZeRO Stage2:梯度分区

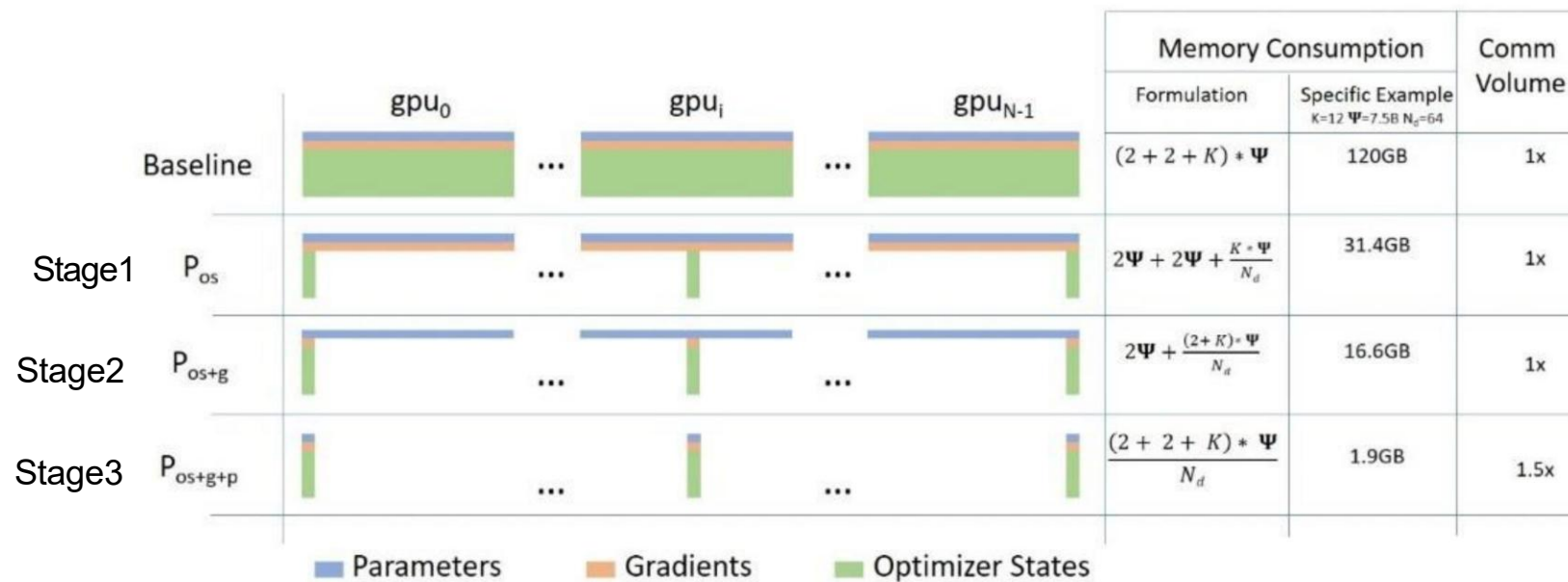


- 跨GPU划分梯度
- 减少负责更新参数的GPU上的梯度

Parameters Gradients Optimizer States

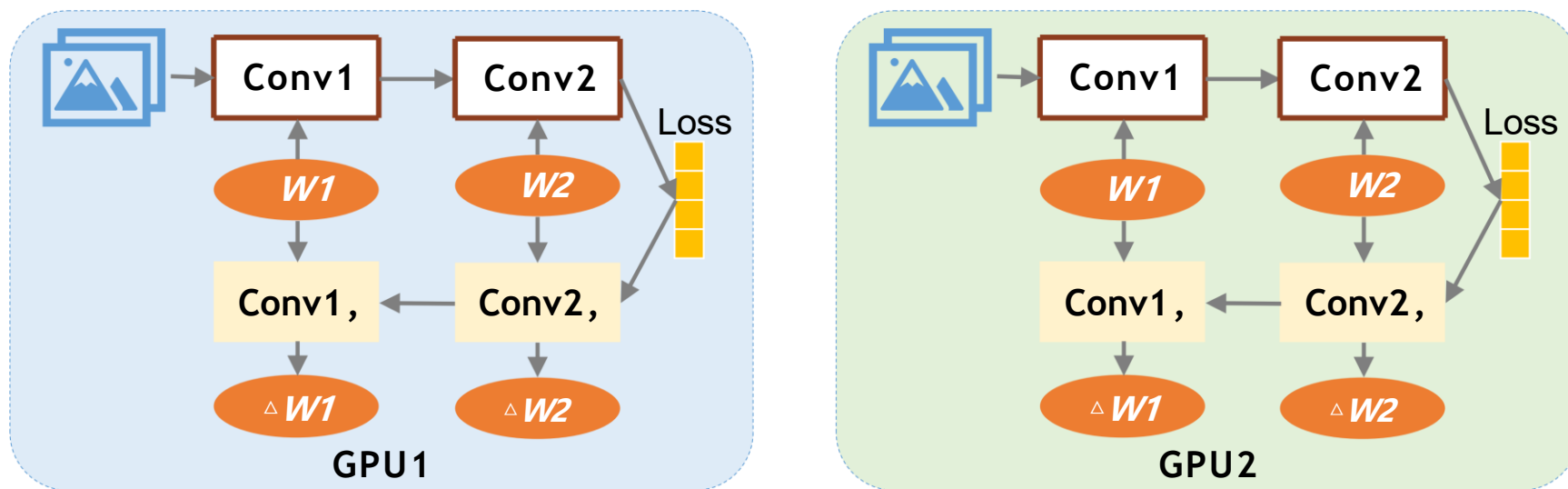
ZeRO:零冗余优化器

- 渐进式内存节省和通信量
- NLR17.2B模型的训练/微调是基于Stage1分布式优化和Megatron框架实现的



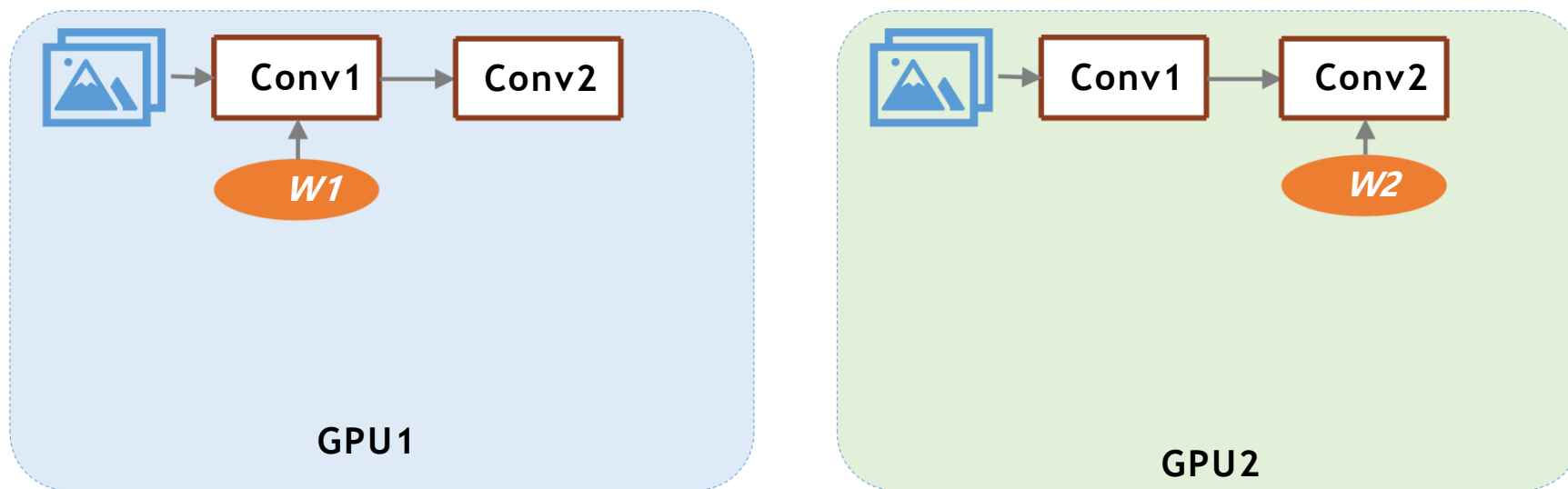
ZeRO Stage3:参数分区

- 在数据并行训练中,所有GPU在训练过程中保留所有参数



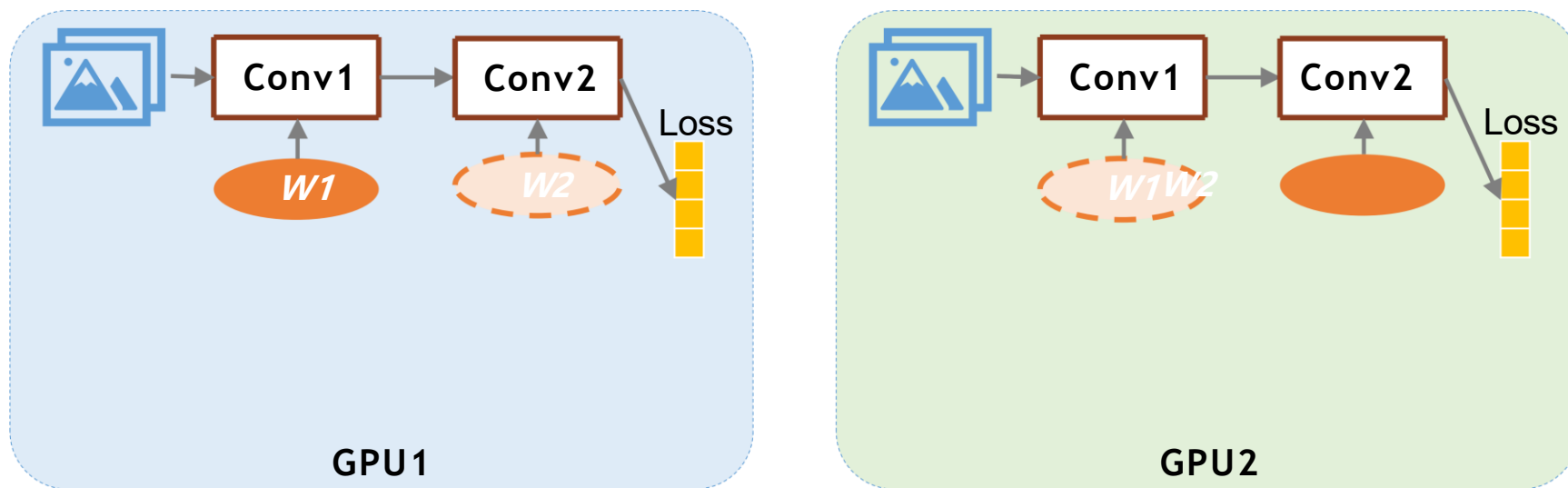
ZeRO Stage3:参数分区

- 在ZeRO中，模型参数在GPU之间进行分区



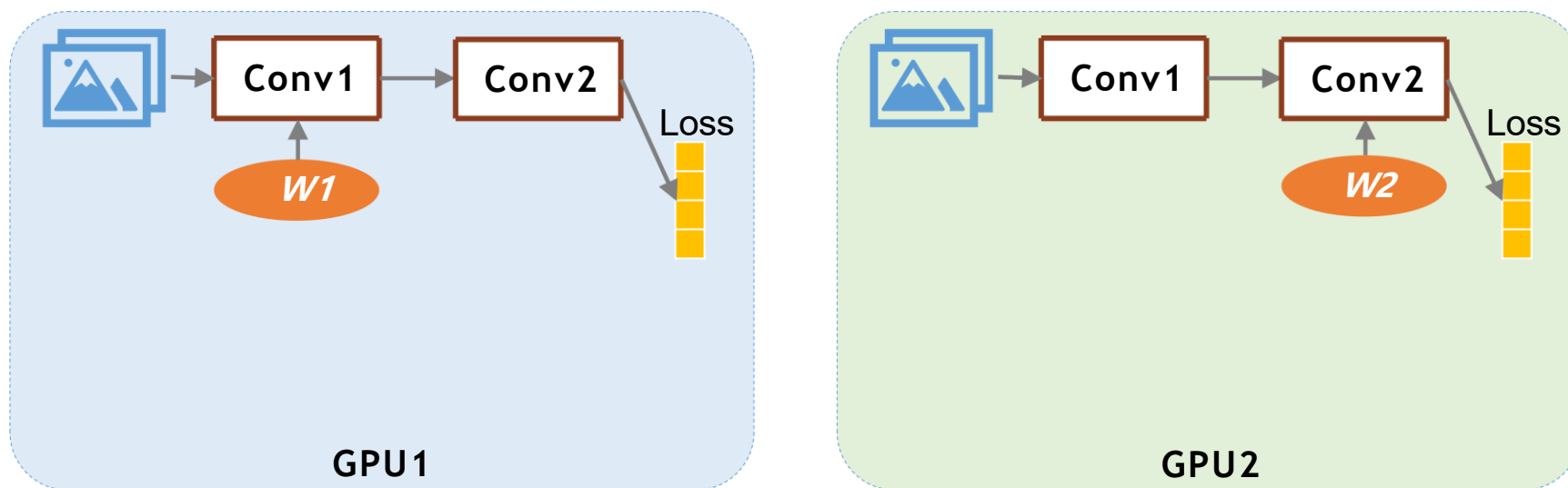
ZeRO Stage3:参数分区

- 在ZeRO中，模型参数在GPU之间进行分区
- GPU在正向传播期间广播其参数



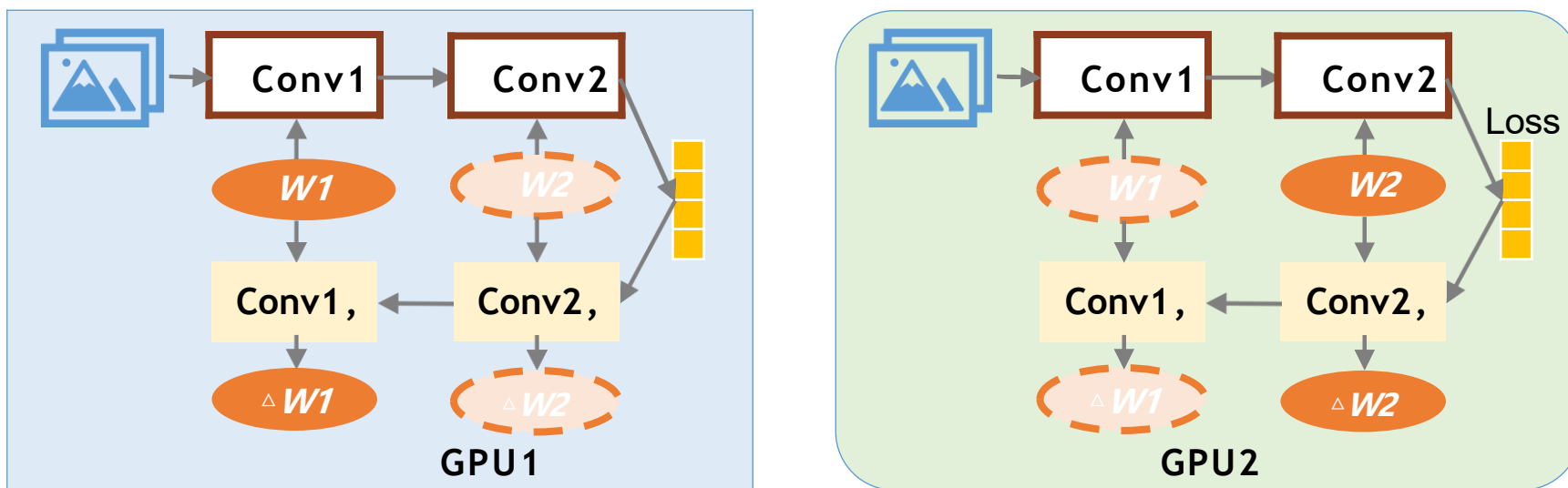
ZeRO Stage3:参数分区

- 在ZeRO中，模型参数在GPU之间进行分区
- 参数使用后立即丢弃



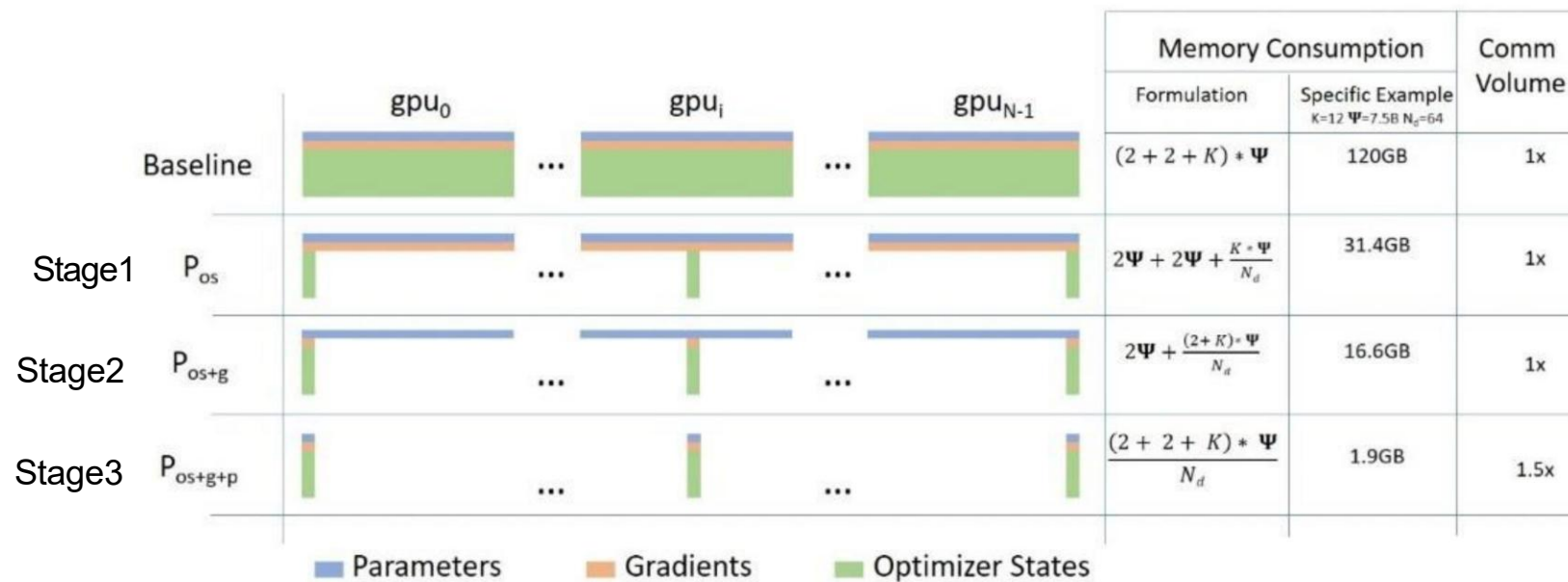
ZeRO Stage3:参数分区

- 在ZeRO中，模型参数在GPU之间进行分区
- GPU在向后时再次广播其参数



ZeRO:零冗余优化器

- ZeRO有三个不同的阶段
- 渐进式内存节省和通信量



3个Stage的选择

小型模型 (<10B参数)：通常只需Stage 1即可解决显存问题，通信开销小，训练效率高

中型模型 (10-30B参数)：建议使用Stage 2，在显存节省和通信开销间取得平衡

大型模型 (>30B参数)：必须使用Stage 3，否则无法在有限显存下训练，可配合CPU Offload进一步降低显存压力

Stage 1：显存节省4-6倍，通信开销中等，适合大多数常规训练

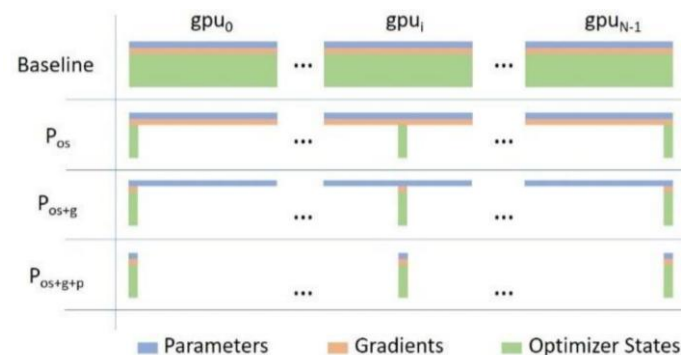
Stage 2：显存节省7-8倍，通信开销较高，适合中等规模模型

Stage 3：显存节省>90%，通信开销高，是训练超大规模模型的唯一选择

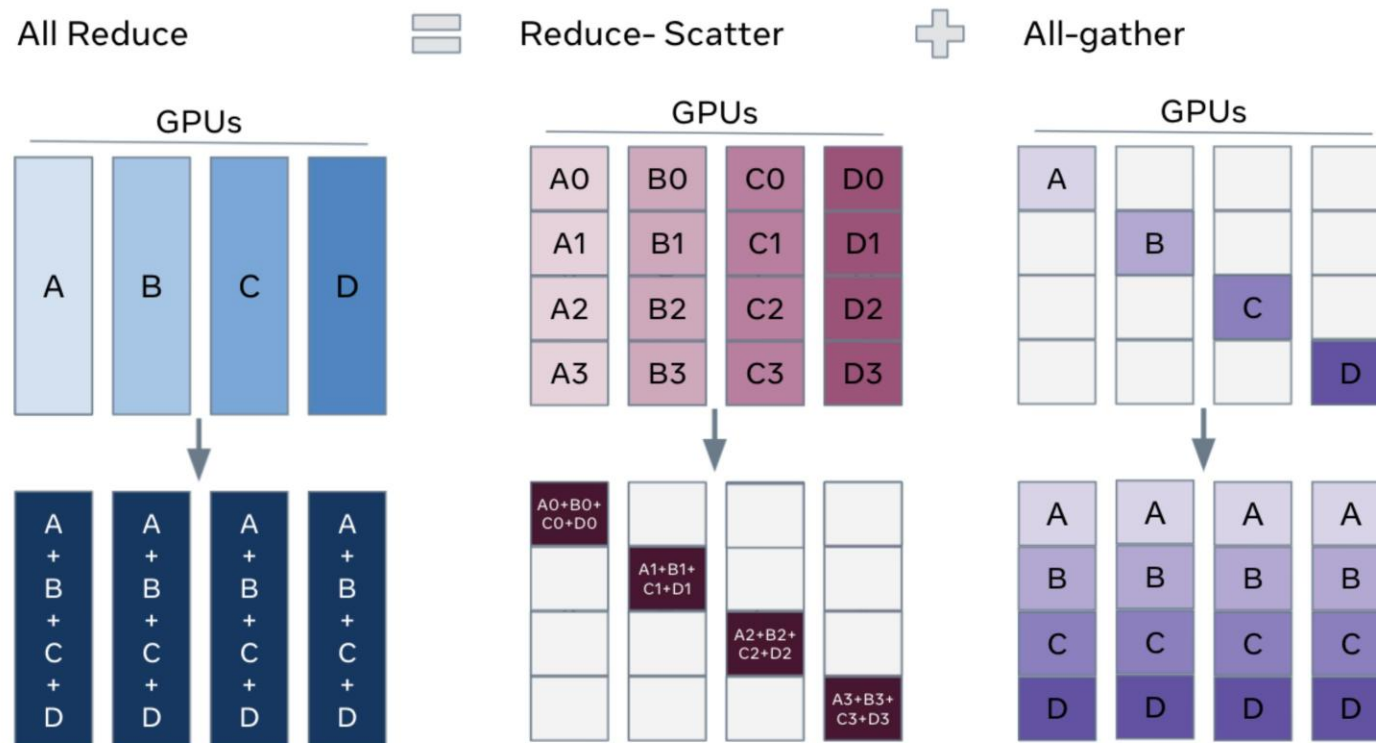
高速网络环境 (InfiniBand/RoCE)：可放心使用Stage 3，通信瓶颈较小

普通以太网环境：建议优先使用Stage 2，避免Stage 3的高通信开销成为瓶颈

消费级显卡 (24GB显存)：必须使用Stage 3+CPU Offload才能训练7B+模型

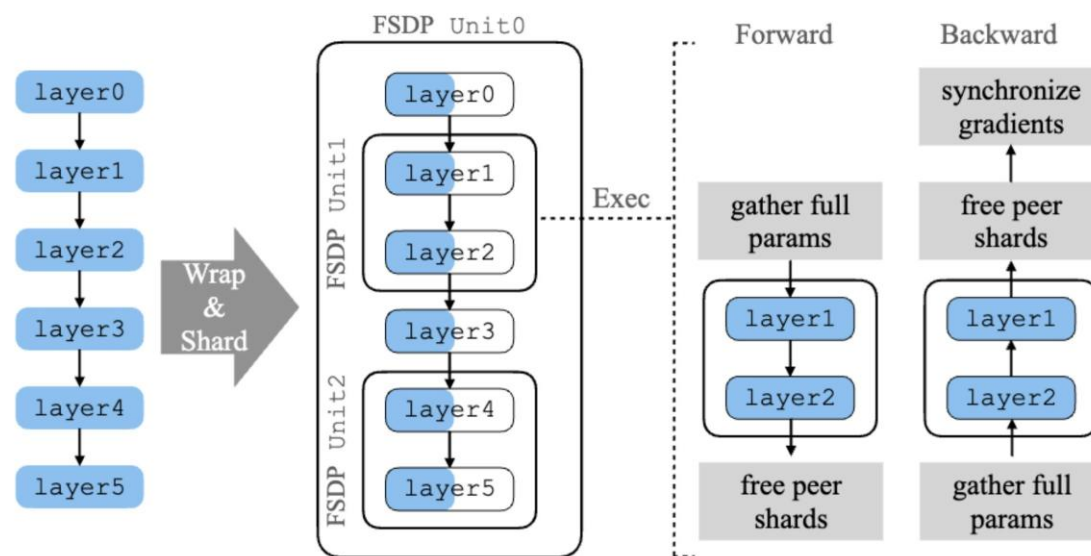


PyTorchFSDP:全分片数据并行



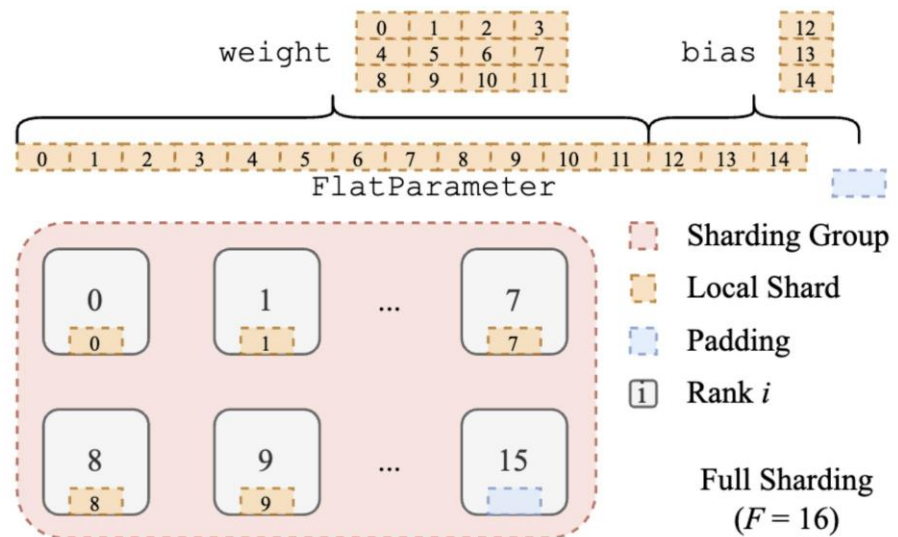
PyTorchFSDP:全分片数据并行

1.将模型参数划分为多个FSDP单元



PyTorchFSDP:全分片数据并行

1. 将模型参数划分为多个FSDP单元
2. 将每个单元分给多个GPUs



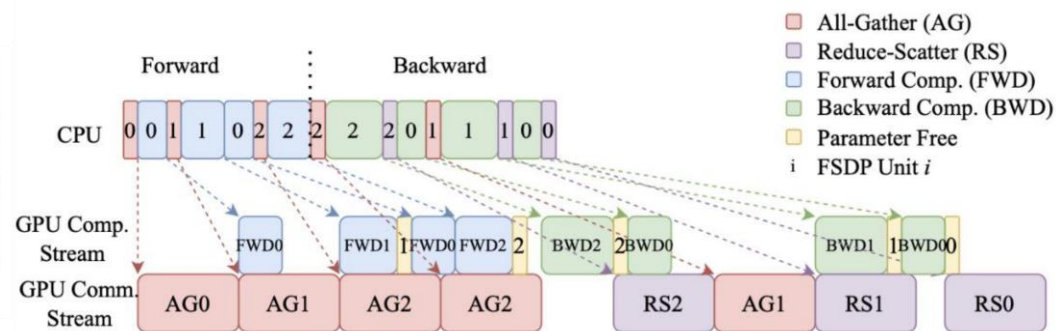
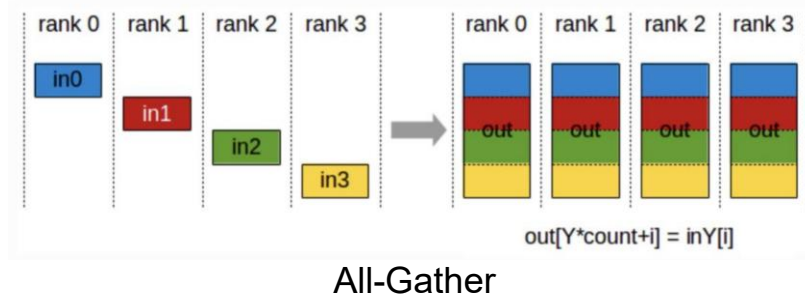
PyTorchFSDP:全分片数据并行

1.将模型参数划分为多个FSDP单元

2.将每个单元分给多个GPU

3.前向传递;

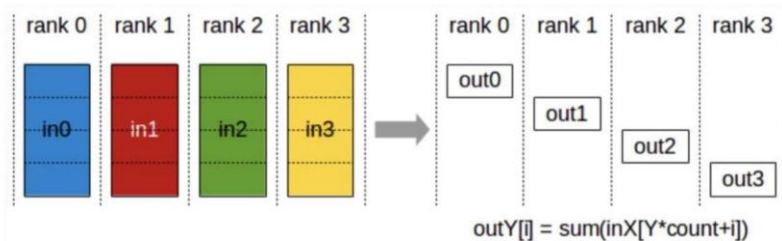
通过all-gather动态重建参数、计算后释放，从而实现参数级别的显存最优分布式训练



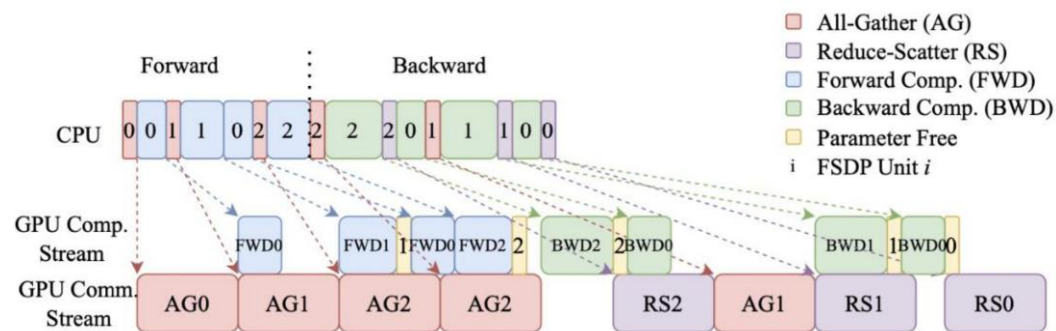
PyTorchFSDP:全分片数据并行

1. 将模型参数划分为多个FSDP单元
2. 将每个单元分给多个GPU
3. 前向传递;
4. 反向传递

1. 再行全集,以取一单位之所有参数
2. 每个GPU计算所有参数的梯度
3. 进行约散射以聚合全梯度



Reduce-Scatter



PyTorchFSDP编程

```
from torch.distributed.fsdp import fully_shard, FSDPModule
from torch.distributed.tensor import DTensor

# 对单个层以及整个模型进行分片 (切分)
model = Transformer()
for layer in model.layers:
    fully_shard(layer)
fully_shard(model)

# 分片之后, 参数会变成 DTensor 类型
for param in model.parameters():
    assert isinstance(param, DTensor)

# 优化器也会自动进行分片
optim = torch.optim.Adam(model.parameters(), lr=1e-2)

# 前向和反向传播过程与普通训练保持一致
for _ in range(epochs):
    x = torch.randint(0, vocab_size, (batch_size, seq_len))
    loss = model(x).sum()
    loss.backward()
    optim.step()
    optim.zero_grad()
```